



UNIVERSITY OF BUCHAREST

**FACULTY OF MATHEMATICS AND
COMPUTER SCIENCE**



COMPUTER SCIENCE SPECIALIZATION

Bachelor's Thesis

THE PAN ARCHITECTURE AS A MODEL FOR FULL-STACK WEB APPLICATIONS

A Case Study on Reducing Food Waste

Graduate: Eduard Emanuel Dinea

Scientific supervisor: Assoc. Prof. Marius Iulian Mihăilescu, PhD

Bucharest, July 2026

Abstract

This thesis proposes and analyzes the **PAN** (**Prisma–Analog–Nx**) architecture, a full-stack model for modern **web** applications built entirely within the **TypeScript** ecosystem, in which the application layers share types and validation schemas derived from a single source of truth. To validate this model, I designed and implemented *Kookio*, a full-stack **web** application that serves as a case study of the **PAN** architecture. *Kookio* addresses a concrete driver of household food waste — buying food without a meal plan — through an integrated flow of pantry management, meal planning and automatic derivation of the shopping list, so that purchases are tied to the planned meals instead of accumulating redundantly.

The architectural motivation stems from the need for **web** stacks that ensure coherence between frontend and backend, as well as a unified development process. **PAN** answers this need not by reducing the number of tools, but by integrating them under a single language and by formalizing the boundaries between components.

Concretely, each layer of this unified **TypeScript** stack covers a distinct responsibility: server-side rendering (**SSR**) is provided by **Angular** through the **Analog** meta-framework, typed client–server communication by **tRPC**, and data persistence by **Prisma** and **CockroachDB**. Organizing the project as a **monorepo** managed with **Nx** enforces an acyclic library topology, checked in continuous integration (**CI**) by the **Nx** `enforce-module-boundaries` rule, and makes it possible to drop the *repository* layer between the **tRPC** procedures and the **Prisma** client.

The implementation is validated through unit tests with **Vitest** and **end-to-end testing (E2E)** with **Playwright**, together with an empirical evaluation of the persistence-layer latencies, front-end performance and test coverage. The measured median latency (warm calls) was 1.71 ms for **CockroachDB** and 0.56 ms for **Redis**, while image optimization reduces the delivered payload by approximately 60 % across the entire catalog. The evaluation shows that the implemented stack achieves performance suitable for a production environment; the coherence and extensibility of the **PAN** architecture, however, are structural properties — supported by the acyclic topology checked in **CI** and by deriving the contracts from a single source of truth —, not consequences of these latency measurements.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Purpose of the thesis	2
1.3	Contributions of the thesis	2
2	Preliminaries	3
2.1	Motivation for the technology choices	3
2.2	The PAN architecture	6
2.3	Technological framework	9
3	Implementation	12
3.1	Implementation overview	12
3.2	Implementation of the application	14
3.3	Functional flows and product algorithms	25
3.4	Methodology and evaluation	29
4	Conclusions	32
4.1	Architectural decisions	34
4.2	Limitations and trade-offs of the architecture	36
4.3	Future directions	37
	Appendices	39
	Appendix A Demonstration of the <i>Kookio</i> application	39
A.1	The main flow: from setup to cooking	39
A.2	Details of the unit conversion pipeline	39
	Appendix B The data model	45
	Glossary of Terms	47
	List of Acronyms	50

1 Introduction

1.1 Motivation

Food waste is a problem with a documented economic and social impact at both global and European level. According to the *UNEP Food Waste Index 2024* report [28], households generated over 631 million tonnes of wasted food in 2022 — the equivalent of more than one billion meals thrown away every day, accounting for 60 % of total food waste worldwide. At the level of the European Union, Eurostat estimates that each inhabitant generated on average 132 kilograms of food waste in 2022, of which 54 % came from households [7]. The European Parliament identifies among the main causes of waste at consumer level: impulse buying, the lack of adequate meal planning and poor management of the household food inventory [6]. These behavioral causes are also confirmed by the academic literature: in a systematic review of 60 empirical studies on household food waste, Schanes et al. [22] identify shopping planning, checking the inventory before shopping and communication between household members as practices that play a decisive role in preventing over-purchasing. The same review reports that simply using a shopping list reduced per-capita waste by approximately 20 %, and explicitly classifies applications that list the food inventory and plan meals among the technological prevention measures. The authors stress, however, that the intention to reduce waste does not automatically translate into behavior (the attitude–behavior gap) and that some of the causes are structural, beyond individual control [22]; consequently, an application such as the one proposed in this thesis can help reduce redundant purchases without being a complete solution to the problem.

In parallel, modern [web](#) applications raise significant technical challenges, documented both in the academic literature and in the annual surveys of the developer community. The *State of JavaScript 2024* survey identifies *excessive complexity*, *option overload* and *breaking changes* as the main pain points reported by users of frontend frameworks [4]. The same source indicates a growing adoption of server-side rendering (SSR) — 59 % of respondents use it in 2024 — and a consolidation of [TypeScript](#) as the standard language for large-scale applications: according to the *JetBrains Developer Ecosystem 2024* survey, [TypeScript](#) adoption grew from 12 % in 2017 to 35 % in 2024, and 67 % of developers write predominantly [TypeScript](#) [11]. These difficulties are amplified in full-stack applications, where poor integration of the layers leads to increased complexity and long-term maintenance costs.

This thesis does not treat these two directions as a relationship of necessity — the food waste problem does not impose a particular architecture —, but uses the first as a vehicle domain for investigating the second: a household application for pantry management, meal planning and shopping lists provides a representative full-stack context (integrated flows, derived data, relational persistence) in which the [PAN](#) architecture can be implemented and evaluated empirically.

1.2 Purpose of the thesis

I set out to define and evaluate the **PAN** architecture, a full-stack **web** model based on the **TypeScript** ecosystem, which integrates the frontend, the backend and the persistence layer into a unified development process. To investigate it empirically, I built *Kookio*, a **web** application that tackles the problem of household food waste. Functionally, the application covers pantry management, daily meal planning and automatic derivation of the shopping list for household users. Thus, *Kookio* serves as a case study for evaluating the **PAN** architecture in a context representative of modern full-stack **web** applications.

To this end, the thesis aims at:

- implementing an integrated functional flow for the pantry, the daily planner and the derived shopping list, with persistence in a relational database;
- integrating the frontend and backend into a unified development process, with shared types and validation mechanisms;
- analyzing the characteristics and structural properties of the **PAN** architecture with respect to modularity (acyclic topology between libraries) and application maintainability (absence of a *repository* layer);
- building a reusable **CI/CD** pipeline, organized into **workflows** and **composite actions**, aligned with the **Nx** targets;
- validating the implementation through unit tests with **Vitest** and **end-to-end testing (E2E)** with **Playwright**, together with an empirical evaluation of the persistence-layer latencies, front-end performance and test coverage.

1.3 Contributions of the thesis

1. **An implemented **PAN** architectural model:** **Analog** (**Angular** 21) with server-side rendering (**SSR**) on top of **Nitro**, **tRPC** for a typed API and **Prisma** to **CockroachDB**, organized as an **Nx monorepo** with four library groups and an acyclic topology enforced in **CI** by the **Nx enforce-module-boundaries** rule.
2. **The *Kookio* application implemented as a **minimum viable product (MVP)**:** a working full-stack **web** application for household users, which covers in an integrated way pantry management, daily meal planning, automatic derivation of the shopping list and the *on-boarding* flow (profile setup), with Firebase authentication, serving as a case study for the **PAN** architecture.

3. **Patterns for sharing data contracts from a single source of truth:** the [Zod](#) schemas are generated automatically from `schema.prisma` by the `prisma-zod-generator` library [5], and the [TypeScript](#) types are derived by composition (`.pick()`, `.omit()`, `.extend()`, `z.infer<>`) from these schemas; the resulting contracts are reused in the user interface, in the server routes and in the tests, with no *repository* layer between the [tRPC](#) procedures and the [Prisma](#) client.
4. **A modular automated CI/CD flow:** based on reusable [workflows](#) and [composite actions](#) aligned with the [Nx](#) targets, so that checks run only for the projects affected by a change (`nx affected`).
5. **A testing methodology on two distinct levels:** unit testing with [Vitest](#) and [end-to-end testing \(E2E\)](#) with [Playwright](#), together with concrete practices for the cold start of the [Vite](#) server (warm-up in `global-setup`) and for navigating to protected routes (`waitUntil: 'commit'`).
6. **An empirical evaluation of the implemented technology stack:** measurements of latency at the persistence layer ([CockroachDB](#) and [Redis](#)), of front-end performance ([Lighthouse](#) and image optimization) and of test coverage, summarized in Table 2.

The application is implemented as a working [MVP](#): recommending recipes based on the pantry inventory is already operational through progressive filtering (Section 3.3.1), while the identified extensions aim at refining this recommendation with the dietary restrictions collected during *onboarding* and at a mechanism for tracking staple consumables. These extensions can strengthen the application’s contribution to reducing food waste by cutting down redundant purchases. The implementation details are presented in Section 3, the empirical evaluation in Section 3.4, and the concrete directions for future development in Section 4.3.

2 Preliminaries

This chapter presents the technological context of the thesis in three complementary subsections. Section 2.1 sets out the motivation behind the technology choices that led to the formulation of the [PAN](#) architecture, Section 2.2 defines the [PAN](#) architecture and compares it with established alternatives, and Section 2.3 describes each component of the stack technically, together with its role in the [Kookio](#) application.

2.1 Motivation for the technology choices

Starting from the advantages of static typing and from the consolidation of [TypeScript](#) as the standard language for large-scale applications — the *JetBrains Developer Ecosystem 2024* survey

shows adoption growing from 12 % in 2017 to 35 % in 2024, with 67 % of developers writing predominantly [TypeScript](#) [11] —, with *Kookio* I explored the possibility of building a complete application, from one end to the other, exclusively in [TypeScript](#). In this thesis, this technology stack will be conventionally called the **PAN** ([Prisma–Analog–Nx](#)) architecture.

The advantages of static typing are not merely an engineering preference; they have been quantified empirically. The study by Gao et al. [8], which evaluated the *Flow* and [TypeScript](#) type systems on a sample of 400 public bugs extracted from real GitHub projects, shows that static typing could have detected approximately 15 % of them (confidence interval [11.5 %, 18.5 %] at the 95 % level). This value is, however, a deliberate underestimate: the authors measure only public bugs, those that survived testing and *review*, and explicitly exclude the collateral benefits of typing — documentation, autocompletion, code navigation and refactoring safety [8]. It is precisely these unmeasured benefits that constitute, in the **PAN** architecture, a central part of the value proposition: renaming a field in `schema.prisma` propagates, at compile time, to every consumer (Section 3.2.4).

The same study also quantifies the cost of typing — the *annotation tax* — at 1.7 and 2.4 tokens per bug for *Flow* and [TypeScript](#), respectively [8] (a *token* being a lexical unit of code — an identifier, keyword or symbol). This low cost is, however, conditioned on the assumption of an already annotated code base, in which the developer incrementally adds only the annotations related to a single change. The **PAN** architecture shifts this trade-off: the type contracts at the data boundary are not annotated by hand, but generated from `schema.prisma` by `prisma-zod-generator` [5]. The base schemas are no longer written by hand, and the only remaining cost at this layer is deriving subsets by composition (`.pick()`, `.omit()`, `.extend()`) — a mechanical operation, requiring significantly less effort than manually annotating each contract. Thus, if the study shows that the benefit of typing holds even under a manual annotation cost, **PAN** largely eliminates that very cost.

An architectural fork that precedes the individual stack choices is the one between an application distributed across microservices and a modular monolith delivered as a single unit. The systematic review of the industrial literature by Soldani et al. [24] finds, based on 51 studies, that the dominant pains of the microservices architecture are intrinsic to distribution: the consumption of network resources (reported by 17 studies as the primary operational pain, since microservices communicate through remote API calls rather than *in-memory*), versioning of API contracts between services, data consistency and distributed transactions (21 and 18 studies, respectively), as well as locating, coordinating and monitoring instances at runtime. Since *Kookio* is a data-centric application, intended for a small team and a single-unit release, I opted for a modular monolith in which these pains do not arise by construction: the [Analog](#) → [tRPC](#) → [Prisma](#) flow executes the calls to the persistence layer in-process, without a *repository* layer and without network boundaries between components (Section 3.2.2). The **PAN** architecture does not reject microservices as an inferior model, but deliberately occupies a different point in the

space of trade-offs. It is important to stress that the cited study does not present the pains of microservices as design flaws, but as intrinsic properties of the complexity of distribution, offset by real gains at large scale — Amazon, Netflix and Spotify being the recurring examples. The present comparison does not establish a hierarchy of value between the two models, but delimits the space of applicability: [PAN](#) is optimal for data-centric applications, monolithic in the sense of single-unit release, developed by small teams — exactly the context in which the pains of distribution outweigh its gains.

Sharing type contracts between the two ends is not, in itself, a novelty: classic distributed systems have long achieved it through interface description languages (*Interface Description Language*, IDL) such as *Protocol Buffers* or *gRPC*, which generate typed code for the client and the server from a common schema. The particularity of the [web](#) ecosystem is that this coherence has historically been hard to achieve: the client and the server often ran in different languages, and the boundary between them — JSON serialization over HTTP — is untyped by nature, so a contract change on the server only surfaced on the client at runtime, as an error. What the [PAN](#) architecture leverages, therefore, is not the invention of type sharing, but the fact that a unified [TypeScript](#) ecosystem — with [tRPC](#), which propagates procedure signatures without a separate IDL, and with [Zod](#) schemas generated from the same base schema — brings this guarantee, already familiar from distributed systems, all the way to the client–server boundary of a web application and checks it at compile time.

Choosing the database

A first conceptual challenge was choosing the database. Historically, *MongoDB* was for many years the default favorite of developers in the JavaScript/[TypeScript](#) ecosystem, thanks to its native JSON storage and its flexible schema that maps directly onto the application’s objects. However, the data model of the [Kookio](#) application is intrinsically relational: recipes, ingredients and users are distinct entities linked through many-to-many relationships, and compound queries — data joins, referential integrity constraints and ACID transactions — are essential for data consistency. This characteristic of the data model made a classic relational database the natural choice for the project.

To abstract the database language I chose [Prisma](#) [19] — an object-relational mapping (ORM) that provides end-to-end type safety in [TypeScript](#), generating the access client directly from the declarative schema. As the concrete database service I turned to [CockroachDB](#) [3], a distributed database compatible with the [PostgreSQL](#) wire protocol [25] — so that [Prisma](#) and the entire tooling ecosystem work unaltered. On top of [PostgreSQL](#), [CockroachDB](#) adds native horizontal scaling, strong consistency through the Raft consensus algorithm, and a serving tier without self-managed infrastructure (*serverless*), suitable for an academic project that may evolve towards a production environment.

The frontend framework and code organization

For the client interface, choosing the [Angular](#) framework came naturally, given the commitment to [TypeScript](#) [10]. The next step was identifying a suitable option for the code that would run on the server. Here I had several valid options, most of them conceptual heirs of the well-known **MERN** architecture (*MongoDB*, *Express*, *React*, *Node.js*), which in recent years has naturally migrated towards stacks based on [Next.js](#) and *Turborepo* (a build system for [monorepos](#)) — both developed by [Vercel](#). However, this direction would have pushed me towards the *React* ecosystem and the *TSX* syntax, abandoning the initial decision to use [Angular](#).

During my research I discovered the [Analog](#) community [1], a [meta-framework](#) that extends [Angular](#) with server capabilities (server-side rendering, file-based routing, API endpoints) — essentially, the [Angular](#) equivalent of [Next.js](#). [Analog](#) benefits from the direct support of the [Nx](#) community [15], a [monorepo](#) management tool developed by the company Nrwl, founded by former engineers of the [Angular](#) team at Google. This lineage explains why [Nx](#) has, at its core, [Angular](#) DNA and why it offers first-class support for [Analog](#) projects: native generators, a distributed cache for build tasks and templates that speed up the creation of libraries dedicated to each core feature of the application.

This is how a structural parallel with the **MEAN** architecture (*MongoDB*, *Express*, [Angular](#), *Node.js*) [12] took shape, with the **PAN** architecture reflecting the maturing of the ecosystem: [Analog](#) absorbs the roles that **MEAN** distributed between *Express* and [Angular](#), and [Nx](#) addresses an architectural concern — [monorepo](#) organization with a distributed cache — that, in the **MEAN** era, was not yet a topic of discussion. This parallel, together with a detailed comparison against the modern *React* stack (**MERN** + [Next.js](#) + *Turborepo*), is presented in Section 2.2.1.

It is worth noting that **PAN** does not answer the complexity pains reported in Section 1 by reducing the number of tools — the stack remains a rich one —, but by integrating them under a single language and by formalizing the boundaries between them. The pain point identified by the surveys is not the number of tools in itself, but the poor integration between layers; **PAN** targets exactly this integration, leaving the complexity structured and automatically checked, not eliminated.

2.2 The PAN architecture

PAN ([Prisma](#)–[Analog](#)–[Nx](#)) is a modular monolithic full-stack architecture for [web](#) applications built entirely within the [TypeScript](#) ecosystem. Its three eponymous components — [Prisma](#) as a declarative [ORM](#), [Analog](#) as an [Angular meta-framework](#) with [SSR](#) support and [Nx](#) as a [monorepo](#) orchestrator — are the building blocks around which a coherent stack is organized according to five fundamental principles:

1. **A single language from end to end.** The entire application — [UI](#), server, validation, persistence — is written in [TypeScript](#). This decision eliminates the class of errors arising from contracts falling out of sync between different languages (for example, Java on the server and [TypeScript](#) on the client) and allows types to be reused uniformly.
2. **A single source of truth for data contracts.** The database schema declared in `schema.prisma` is the primary source from which both the [Prisma TypeScript](#) client and the [Zod](#) validation schemas (via `prisma-zod-generator`) are automatically derived. The application types are then composed from these schemas with the `.pick()`, `.omit()`, `.extend()` and `z.infer<>` operators. A change in `schema.prisma` propagates, at compile time, through all the layers.
3. **An acyclic modular topology, enforced in CI.** The code is organized as an [Nx monorepo](#) with four library groups (`feature`, `prisma`, `server`, `shared`) and a strictly acyclic dependency relationship. The [Nx](#) `enforce-module-boundaries` rule rejects in continuous integration any import that violates the topology, guaranteeing that the architectural structure does not degrade as the project evolves.
4. **No intermediate *repository* layer.** The [tRPC](#) procedures call the [Prisma](#) client directly (`ctx.prisma`), without an intermediate object-oriented *repository* layer. This decision reduces the number of adaptation layers and forces the business logic to be expressed in terms of the primitives exposed by [Prisma](#) — an explicit trade-off between simplicity and abstraction.
5. **Server-side rendering as the default mode.** The [Angular](#) + [Analog](#) combination delivers [SSR](#) as the default behavior for all routes. As a result, the user’s first impression relies on HTML that is populated from the very first request, which matters both for [SEO](#) indexing and for perceived performance.

2.2.1 Comparison with the modern React stack

The contemporary React stack (MERN + [Next.js](#) + [Turborepo](#)) occupies the same solution space as [PAN](#), but in a different ecosystem. Figure 1 visually summarizes the maturing from [MEAN](#) to [PAN](#), and Table 1 extends the comparison to the modern React stack.

The major differences between [PAN](#) and the modern React stack are:

- **Community.** [PAN](#) is built on [Angular](#), a framework with a smaller usage share than React (50.1 % versus 81.1 %, *State of JavaScript 2024* [4]). This difference is both a practical limitation (fewer available developers, fewer *third-party* [UI](#) libraries) and an invitation to discipline — [Angular](#) imposes a more structured organization by default, through dependency injection and strictly typed *standalone* components.

Architectural evolution: MEAN → PAN

Structural parallel between the traditional JavaScript stack (2013) and the proposed architecture (2026)

ARCHITECTURAL CONCERN	MEAN (2013)	PAN (proposed)
Frontend framework	AngularJS	Analog <i>unifies frontend + server</i> (via Vite + Nitro)
Server framework	Express.js	
Execution runtime	Node.js	Node.js
Data access (ORM)	Mongoose <i>optional - not in the acronym</i>	Prisma
Database	MongoDB	CockroachDB <i>compatible with the PostgreSQL wire protocol</i>
Monorepo management	<i>not a concern in the MEAN era</i>	Nx

OBSERVATIONS

- **Analog** absorbs the roles that MEAN distributed between Angular and Express, reflecting the architectural compression of modern meta-frameworks.
- **Nx** addresses a concern (monorepo management with a distributed cache) that was not a topic of discussion in the MEAN era.
- **Prisma + CockroachDB** replace MongoDB with a relational stack that keeps document-style flexibility through the JSONB type with GIN indexing.

Figure 1: Structural parallel between the **MEAN** architecture (Karpov, 2013) and the **PAN** architecture proposed in this thesis. The diagram highlights the maturing of the ecosystem: **Analog** absorbs the roles distributed in **MEAN** between *Express* and *Angular*, and **Nx** addresses the concern of **monorepo** organization which, in the **MEAN** era, was not yet a topic of architectural discussion.

Table 1: Comparison between **MEAN** (2013), the modern React stack and the **PAN** architecture.

Aspect	MEAN (2013)	MERN + Next.js + Turborepo	PAN
Frontend	Angular (legacy)	React	Angular 21
Meta-framework	— (Express)	Next.js	Analog
Server runtime	Node.js + Express	Node.js + Next.js	Nitro
API	Manual REST	API routes + optional tRPC	Mandatory tRPC
Shared types	No	Partial (App Router)	Automatic end-to-end
Database	MongoDB	MongoDB or PostgreSQL	CockroachDB
ORM	Mongoose	Prisma or Drizzle	Prisma
Monorepo	—	Turborepo	Nx
Language	JavaScript	Partial TypeScript	Pure TypeScript

- **Maturity.** [Next.js](#) has almost a decade of evolution and extensive production adoption; [Analog](#) is under three years old and has a small community. [PAN](#) compensates by choosing [Angular](#) and [Nitro](#) (both mature individually) as base platforms, but accepts the community risk of [Analog](#).
- **End-to-end typing.** In [PAN](#), [tRPC](#) is a mandatory component of the architecture, not an optional one. This decision makes [PAN](#) more restrictive than the modern React stack (where you can choose REST, GraphQL or [tRPC](#) ad hoc), but eliminates the class of contract desynchronization errors between client and server.

2.3 Technological framework

2.3.1 Modern web architectures

For a cooking application in which the user's first impression depends on HTML populated from the very first visit, the client-side rendering ([CSR](#)) model of a classic [Angular](#) single-page application ([SPA](#)) was not enough. The requirements of fast display, [SEO](#) for public recipes and subsequent on-page interactivity led to an architecture that splits the [rendering](#) work between the server and the client [[1](#), [29](#)].

The [rendering](#) strategies relevant to the project are:

- **CSR** (client-side rendering) — used in [Kookio](#) for the authenticated screens (planner, pantry), where interactivity comes first;
- **SSR** (server-side rendering) — for public pages with dynamic content (recipe feed, profiles), where the initial HTML must be complete;
- **SSG** (static site generation) — prerendering, at *build* time, the pages whose content is completely stable; it is supported by [Analog](#), but not used in the current version of the application, in which static pages are server-rendered as well;
- **hybrid architectures** — combining these strategies on different routes, depending on the nature of the content [[1](#)].

I chose a hybrid full-stack architecture in which the frontend and the backend run in the same [TypeScript](#) code base. The decision is motivated practically: the types and validation schemas ([tRPC](#) with validation through [Zod](#) schemas) are shared at module level, and a single build step delivers both layers [[27](#), [33](#)].

2.3.2 Angular and server-side rendering

I chose [Angular](#) as the frontend framework because its ecosystem natively provides the pieces I needed for *Kookio*: strict typing, *standalone* components, [Angular](#) Signals (reactive signals) and the [Angular](#) Material/CDK component set, consistent with respect to accessibility [10, 9].

SSR support in *Kookio* comes from [Analog](#), a [meta-framework](#) that extends [Angular](#) with:

- [SSR](#) (server-side rendering) and [SSG](#) (static site generation) configurable per route,
- routing based on the file structure,
- integration of the server logic ([Nitro](#), [tRPC](#)) in the same project [1, 2, 29].

For a public recipe page, [SSR](#) makes the difference between a minimal HTML document until [hydration](#) and HTML that is populated from the very first request — important both for the user and for the [SEO](#) indexing on which the organic distribution of the content relies [1].

2.3.3 Monorepo and orchestration with Nx

I organized *Kookio* as a [monorepo](#) using [Nx](#): a single application (apps/web) and a set of libraries segmented by responsibility (feature, prisma, server, shared) [15].

The libraries are grouped by role:

- `feature/` — user flows (auth, shell),
- `prisma-client` — the schema, the generated client, the [Zod](#) schemas,
- `server/` — the [tRPC](#) context, logging, middleware, server runtime utility functions,
- `shared/` — [UI](#), utilities, simulated (*mock*) data.

On top of this structure, [Nx](#) provides two essential mechanisms used systematically in the project: `nx affected` runs only the tasks (lint, test, build) touched by the current changes, and the `enforce-module-boundaries` rule rejects in [CI](#) the imports that violate the library topology [15].

2.3.4 tRPC and type sharing

Client–server communication in *Kookio* uses [tRPC](#) (*TypeScript Remote Procedure Call* — typed remote procedure calls for [TypeScript](#)): the procedures are declared once, in [TypeScript](#), on the server, and the [Angular](#) client invokes them directly, without an additional layer of API definitions [27].

The concrete advantages I observed:

- full [TypeScript](#) inference eliminates the hand-written [DTOs](#) that appear in classic REST architectures,
- input validation is done through [Zod](#) schemas generated automatically from the [Prisma](#) schema [33, 5],
- refactorings that rename fields propagate instantly across both layers.

For [Kookio](#), in practice, this translated into the almost complete elimination of type checks at the client–server boundary: the contract is the compiler.

The limits of pure static typing also motivate the runtime validation layer. The taxonomy of undetectable bugs in [8] ranks string-content errors (*StringError* — malformed URLs, malformed queries) second by frequency, since the `string` type is opaque to most static type systems. Validation through [Zod](#) schemas at the [tRPC](#) boundary covers precisely part of this gap, checking at runtime formats, enums and constraints that the [TypeScript](#) compiler cannot express.

2.3.5 Persistence: Prisma and CockroachDB

For persistence I chose [CockroachDB](#) as the database and [Prisma](#) as the [ORM](#) [19, 3]. [CockroachDB](#) is compatible with the [PostgreSQL](#) wire protocol, which means that the SQL tooling ecosystem (drivers, `psql` clients, [Prisma](#)) works without adaptations, while additionally offering native horizontal scaling capabilities. Standard [PostgreSQL](#) focuses on vertical replication, whereas [CockroachDB](#) adds native horizontal scaling on top of the protocol [3].

In the project, [Prisma](#) covers three responsibilities: declaring the schema in `schema.prisma`, generating a [TypeScript](#) client with exact types for each model, and managing the versioned migrations applied at `deploy` [19].

To generate the [Zod](#) schemas I use `prisma-zod-generator`, which exposes a validatable copy of each model — the source of truth is the [Prisma](#) schema, and the API and [UI](#) layers consume derivatives of it [5].

2.3.6 Automated testing: Vitest and Playwright

The test suite of [Kookio](#) has two distinct layers: unit tests with [Vitest](#) for isolated logic and end-to-end testing (E2E) with [Playwright](#) for complete scenarios through a real browser [32, 13].

I chose [Vitest](#) because it already runs on [Vite](#) (the same bundler as the main application), which means there are no duplicated configurations or divergences between `nx test` and `nx build` [[32](#), [31](#)].

[Playwright](#) covers the critical flows: authentication, completing *onboarding*, adding to the pantry, viewing the planner. The suite runs in [CI](#) on every [PR](#) targeting test before promotion to main [[13](#)].

2.3.7 Configuration and environment management

Environment variables (Firebase keys, [Redis](#) URLs, [Vercel](#) secrets) are defined only once in the [Vercel](#) dashboard, separately per environment (development, preview, production) [[30](#)]. This centralization eliminates `.env` files accidentally added to the Git history and keeps all branches in sync.

Locally, synchronization is done with `vercel env pull`, which writes a `.env.local` (excluded from version control through `.gitignore`) with the values of the current environment — no secret is versioned in Git [[30](#)].

3 Implementation

3.1 Implementation overview

The previous chapter established the technology choices that make up the [PAN](#) architecture (defined in Section [2.2](#)). In what follows I describe how this pattern translates into the actual structure of the repository: a single [Analog](#) application under `apps/`, a set of [Nx](#) libraries grouped by responsibility under `libs/`, and a unified execution chain through which a request travels from the [Analog](#) server-side runtime to the [tRPC](#) and [Prisma](#) layers.

3.1.1 Structure of the Nx monorepo

```
kookio/
├── apps/
│   ├── web/                # Analog application (Angular 21 +
│   │   │                   SSR),
│   │   │                   runs both in the browser and as
│   │   │                   a server runtime (Nitro)
│   │   └── web-e2e/        # Playwright suite for the critical
│   │                       flows (login, onboarding, pantry,
│   │                       planner)
├── libs/
│   ├── feature/            # user-facing flows
│   │   ├── auth/          # login, registration, session
│   │   ├── onboarding/    # profile setup steps
│   │   ├── pantry/        # pantry management
│   │   ├── plan/          # daily planner
│   │   ├── recipes/       # recipe feed and details
│   │   ├── groceries/     # derived shopping list
│   │   ├── saved/         # saved collections
│   │   ├── dashboard/     # main authenticated screen
│   │   └── shell/         # layout, topbar, navigation
│   ├── prisma-client       # Prisma schema, migrations,
│   │                       generated Zod schemas, derived types
│   ├── server/            # auth, logger, trpc
│   │                       (server runtime helpers)
│   └── shared/            # ui, utility, mock
│                           (components, helpers, fixtures)
├── nx.json                # global Nx configuration
├── package.json           # main project manifest
├── tsconfig.base.json     # TypeScript config + path aliases
├── vercel.json            # Vercel routes, headers and build
└── vitest.workspace.ts    # Vitest workspace (all projects)
```

In practice, a change in `libs/feature/pantry` triggers only the checks for the affected library and for the modules that depend directly on it (via `nx affected`), not for the entire repository [15]. The topology between groups (`feature/` does not import from `feature/`, `shared/` depends on nothing internal) is enforced automatically by the `enforce-module-boundaries` rule, checked in [CI](#).

3.1.2 End-to-end application flow

At a high level, a request goes through three layers: *Analog* (*UI and SSR*) → *tRPC* (*typed API*) → *Prisma* (*data access*) [1, 27, 19]. The transitions between layers do not go through an additional

layer of manual serialization — the [TypeScript](#) types are shared from end to end, and input validation at the [tRPC](#) boundary uses [Zod](#) schemas derived from the [Prisma](#) schema [27, 33, 5]. The step-by-step breakdown of a concrete request (from the initial [SSR](#) to the subsequent calls from the browser) can be found in Section 3.2.9, and the corresponding sequence diagram in Figure 4. The overall picture of the components and of the externally managed services is shown in Figure 2.

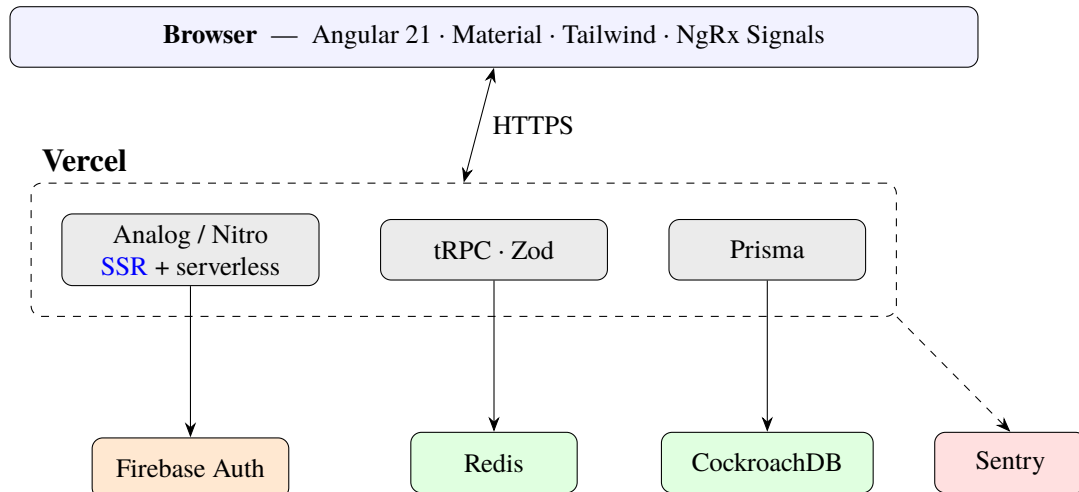


Figure 2: Overall architecture: the browser ([Angular](#)) communicates over HTTPS with the [Vercel](#) platform, which hosts the [Analog/Nitro](#) runtime ([SSR](#) + serverless functions), the [tRPC](#) layer (with [Zod](#) validation) and the [Prisma](#) client. Persistence and cross-cutting services are managed externally: [CockroachDB](#) (data), [Redis](#) (cache, *rate-limiting*), [Firebase](#) (authentication) and [Sentry](#) (error monitoring).

3.2 Implementation of the application

Whereas the previous section justified the technology choices, this one shows how each choice looks in the repository: which files materialize it, which conventions apply it and what concrete constraints it imposed at the code level. For each technology, the emphasis is no longer on *why* it was chosen, but on *what it actually produced* once integrated into [Kookio](#).

3.2.1 Modern web architectures

The [rendering](#) configuration of [Kookio](#) is concentrated in `apps/web/vite.config.ts`: the [analog](#) plugin is called with `ssr: true`, with no prerendered routes. Thus, [SSR](#) is the rendering mode for all the application’s routes, including pages with stable content such as `/about`; [SSG](#), although supported by [Analog](#), is not used, so that every response goes uniformly through the server runtime [1].

On the [SSR](#) path, the distinction between public routes (`/recipes`, `/legal/attributions`) and authenticated ones (`/pantry`, `/plan`) does not show up in the

rendering strategy, but in what the server executes before serving the page: authenticated routes go through a *guard* (route interceptor) that issues a redirect to `/login` if the Firebase session is missing, visible to [Playwright](#) as `ERR_ABORTED` on Chromium and handled through `waitUntil: 'commit'` in the [E2E](#) tests.

3.2.2 Library topology and the absence of a repository layer

The application's *feature* libraries are vertical slices: each one represents a complete user flow (auth, onboarding, pantry, plan, recipes) and contains the components, the state stores (NgRx Signals *stores*) and the services that flow depends on. The topology enforced by the [Nx](#) `enforce-module-boundaries` rule is acyclic: `apps/` → `libs/feature/` → `libs/server/`, `libs/shared/`, `libs/prisma-client`, and `libs/feature/X` cannot import from `libs/feature/Y`. Communication between features happens through `providedIn: 'root'` state stores or through the [tRPC](#) procedures exposed by `libs/server/trpc` [15, 14, 27]. The project's dependency graph, collapsed onto the four *scope* domains, is shown in Figure 3.

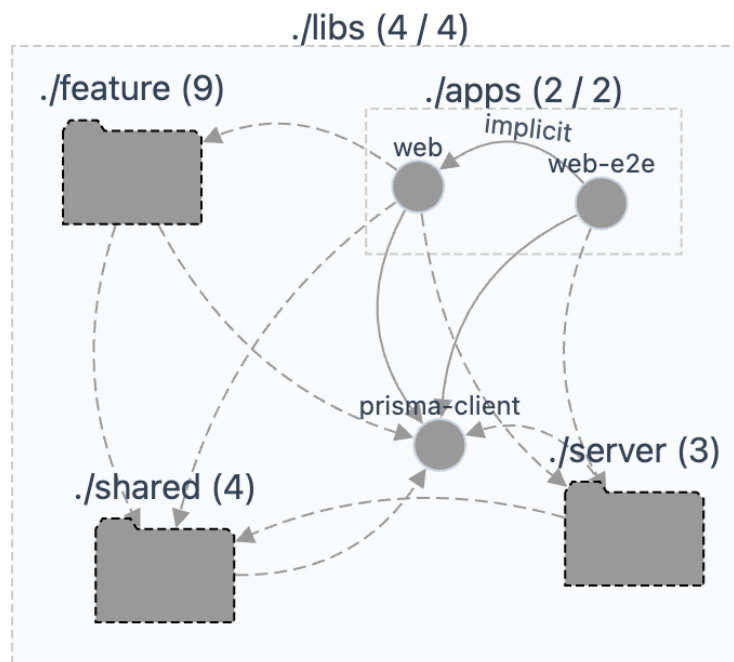


Figure 3: The project's dependency graph ([Nx](#)), collapsed onto the four *scope* domains: feature (9 libraries), server (3), shared (4) and `prisma-client`, plus the `web` and `web-e2e` applications. The arrows show the dependency relationship; the topology is acyclic (`apps` → `feature` → `server`, `shared`, `prisma-client`), with the lower layers having no dependencies on the upper ones.

Organizing the feature libraries as vertical slices (each a complete user flow — auth, pantry, plan), rather than as horizontal layers matching the processing stages, reflects Parnas's central

conclusion: it is almost always incorrect to begin the decomposition of a system on the basis of a flowchart [16]. Parnas contrasts the two modularizations of the KWIC system — one by stages (input → circular shift → alphabetizing → output), the other by information hiding — and shows that in the first a change such as the storage format propagates into every module, whereas in the second it stays confined to a single one. Because design decisions transcend the time order of execution, modules do not correspond to processing steps. In PAN, this observation translates into the structural prohibition `feature/X ↗ feature/Y`: slices communicate through `tRPC` procedures or root-level state stores, not horizontally through stages.

The acyclic topology enforced by the `Nx enforce-module-boundaries` rule also corresponds to the `USES` relation that Parnas requires to be a loop-free partial ordering [16]. The benefits he attributes to this ordering can be found in *Kookio*: the lower layers (shared, server, prisma) can be used without the upper ones, and “cutting off” the upper features leaves a reusable trunk — exactly the property that allows a new feature library to be added incrementally without touching the rest of the application. It is important, however, to mark the distinction Parnas makes explicitly: a hierarchical structure (an acyclic `USES` relation) and a “clean” decomposition (through information hiding) are two desirable but independent properties. The `Nx enforce-module-boundaries` rule guarantees the former — the dependency graph stays acyclic —, but does not by itself guarantee that each library actually hides a changeable design decision. The latter remains a discipline applied when drawing the boundaries: the `CI` rule prevents the topology from degrading, not the absence of a real secret in a poorly delimited library.

The backend and frontend code coexist in the same repository: the `Prisma` client and the generated `Zod` schemas live in `libs/prisma-client`, and their types are imported directly into the `Angular` components [5]. There is no classic object-oriented *repository* layer between the `tRPC` procedures and `Prisma`; the routers call `ctx.prisma` directly, and the multi-step transactions — including writes that touch several domains, such as the atomic transfer of bought items from the shopping list into the pantry (`groceries.addBoughtToPantry`, Section 3.3.4) — use `prisma.$transaction` [27, 20]. This direct coupling does not reduce the testability of the procedures: the `tRPC` context remains an injection point through which `ctx.prisma` can be replaced with a test double, a mechanism detailed in Section 3.2.10.

3.2.3 Angular and server-side rendering

The application’s routes are defined based on the file structure (*filesystem-based*) under `apps/web/src/app/pages/`: a file `about.page.ts` exposes `/about`, `cook.[recipeId].page.ts` exposes a dynamic route with a parameter, and a co-located `cook.[recipeId].server.ts` loads data before `Angular` renders the component [1, 2]. Each `*.page.ts` is a *default export* and is loaded on demand (*lazy loading*) by the `Analog` router; there is no centralized routes file.

The application-level [SSR](#) configuration is minimal: `app.config.ts` registers file-based routing, the HTTP client and hydration with replay of the events captured in the interval between [SSR](#) and [hydration](#) (the concrete mechanism is described in Section 3.2.9) [1]. Each import added to `app.config.ts`, however, carries a cost amplified by [SSR](#): the module is evaluated on the critical path of every route, and aggregating *barrel* files (re-exports of the `export *` kind) are not eliminated by [Vite](#) in the development environment. The practical constraint is to import directly, through sub-paths, only the tokens actually used, leaving heavy components (Material datepicker, autocomplete) to be loaded from the level of the page that uses them.

3.2.4 Data management with Prisma and CockroachDB

The database schema lives at `libs/prisma-client/schema.prisma`, alongside the migrations managed by [Prisma](#) and the `prisma-zod-generator` generator, which produces a validatable copy of each model in [Zod](#) form [19, 5]. The complete schema — the 20 models and the relationships between them — is shown as an entity-relationship diagram in Appendix B. The derived types (initial data sets, form inputs, payloads for [tRPC](#) routes) are built by composition — `.pick()`, `.omit()`, `.extend()` — directly on top of the generated schemas, so that renaming a field in `schema.prisma` propagates automatically to the consumers [5].

Two structural decisions support this arrangement. First, the public `@kookio/prisma-client` entry point is safe to run in the browser: it exports only types and [Zod](#) schemas, while the `PrismaClient` instance is isolated under the `@kookio/prisma-client/server` sub-path, whose import is blocked by ESLint in frontend code [19]. Second, the [Prisma](#) instance is extended with an automatic *retry-on-conflict* for the [CockroachDB](#)-specific *serializable* error codes (P2034, P2036, 40001 and the *restart transaction* message); all write operations (*create*, *update*, *upsert*, *delete* and the **Many* variants) go through this *wrapper*, and the multi-step transactions are designed to be idempotent so that they can be retried automatically [20, 3].

The concrete connection to [CockroachDB](#) is made through the *driver adapter* pattern introduced in [Prisma](#) 7: the `PrismaClient` instance receives at construction an explicit adapter (`@prisma/adapter-pg`) that encapsulates the *pg* connection *pool* for the [PostgreSQL](#) wire protocol [19, 3]. The connection pool is sized for the *serverless* execution model: at most five connections per instance, with a connection *timeout* of 10 seconds and a query *timeout* (`statement_timeout`) of 20 seconds, so that an ephemeral instance does not exhaust the database's connection quota and a stuck query does not hold a connection indefinitely. Starting with this version, [Prisma](#) drops the native engine written in Rust: the client runs entirely in JavaScript, and the database URL is moved from `schema.prisma` into a dedicated `prisma.config.ts` file. This separation between the typed client (generated from the schema) and the actual transport driver makes it possible, in principle, to change the data source without regenerating the client — a flexibility

that is useful for integration testing against a local database and for future migrations between PostgreSQL-compatible providers.

Compound uniqueness constraints are declared in `schema.prisma` and enforced at the database level — for example `@@unique([userId, date, slotType])` on `MealSlot`, or the `userId-ingredientCatalogId` pair on `PantryItem` —, and [Prisma](#) automatically generates the *compound key* forms (of the kind where: `{ userId_ingredientCatalogId: { ...} }`) for the corresponding lookup queries [19].

3.2.5 Building the initial data set

The data the database is populated with — the catalog of ingredients, categories, culinary areas and the 594 recipes — is not written by hand, but derived from a public source, *TheMealDB*, through an offline fetching and cleaning pipeline. The source, however, is a *crowd-sourced* database: the instructions are free text, the measures appear in heterogeneous forms ("2 tbsp heaped", "1/2 cup"), and the ingredient names contain duplicates, plural forms and typos. Cleaning this data into a form that conforms to the [Prisma](#) schema requires deterministic, versioned and reproducible transformations — which is why the pipeline is built as a set of [TypeScript](#) scripts separate from the application, under `research/the-meal-db/`, each one consuming the artifact of the previous phase and producing a JSON *snapshot*.

The pipeline runs in five phases. The *fetching* phases query the source API — the catalog (categories, areas, ingredients) and the recipes, the latter discovered by scanning the 26 letters of the alphabet, with up to 20 ingredient positions parsed for each recipe from the source's flat structure. The *catalog cleaning* phase normalizes the names to a stable `snake_case` key (diacritics removed, whitespace collapsed) used as a unique identifier throughout the pipeline, and auto-classifies each ingredient into one of the categories through an ordered list of twelve matching rules (*first-match-wins*, with `PRODUCE` as the final catch-all rule; ingredients that do not match remain unclassified, for manual review). The *recipe cleaning and enrichment* phase derives the structured fields (described below), and a final *residual fixes* phase applies, directly to the canonical files, a set of idempotent mechanical rewrites.

One structural decision of the pipeline deserves highlighting: the separation between the automatic output and the canonical source. The cleaning scripts never overwrite the `recipes.json` file committed to the repository — they write to an intermediate snapshot (`clean-recipes.json`), and promotion into the canonical source happens through a deliberate manual review step. As a result, hand-made refinements (re-segmentations of steps that the automatic parser missed, transcription fixes) survive re-running the pipeline, while purely mechanical fixes remain automated and idempotent. The source of truth is versioned, not regenerated on every run.

Three of the enrichment transformations are heuristics with an algorithmic role, not mere reformatting. Estimating the *number of servings* maps each ingredient to its mass in grams — volumes through the density of water, piece-type units through a table of per-piece weights — excluding pure flavoring ingredients (salt, oil, vinegar) and negligible units; the total bulk mass is divided by a tunable constant of 400 g per serving — an indicative weight for a main-course serving, chosen pragmatically and tunable, with no claim to nutritional value — and clamped to the interval $[1, 12]$, with a default of 2 servings for recipes with fewer than two bulk ingredients or under 200 g of usable data. *Difficulty* is a three-tier *bucket* (EASY, MEDIUM, HARD) derived from a complexity score of $\text{ingredients} + 2 \times \text{steps}$ — steps weighted double, since they reflect effort, not just the shopping list — relative to the median of the scores across the entire catalog: below $\text{median} \times 0.8335$ is EASY, above $\text{median} \times 1.1665$ is HARD, the rest MEDIUM (the bands are the median $\pm 1/6$, so they yield an even split only for a uniform distribution; since recipe complexity is right-skewed, there are more EASY than HARD recipes). Finally, *measure parsing* turns the free-text strings into (quantity, MeasureUnit, notes) triples — ranges take the lower bound, and unrecognized units are moved into the notes field, without losing information.

The residual fixes phase is where the reality of crowd-sourced data shows most clearly: twenty-four distinct transformations, each idempotent (run a second time, it changes nothing). Rather than an enumeration, they fall into four classes of data quality issues: *typographic and Unicode normalization* (typographic apostrophes and curly quotes brought to ASCII, Unicode fractions parsed, zero-width characters removed); *referential integrity* (rewriting ingredient names from plural to singular according to the merge plan, so that recipes link correctly to the catalog, plus deduplicating ingredients repeated within the same recipe, which the `@unique([recipeId, ingredientCatalogId])` constraint would reject); *reconstruction of the step structure* (splitting steps that contained internally numbered sub-steps, removing header steps and metadata leaked as instructions); and *regeneration of derived fields* (kebab-case slugs from the title, with explicit mapping of extended Latin characters, and reclassifying difficulty after the step restructuring shifted the global median). Idempotence is guaranteed structurally — each transformation matches only the uncorrected form or stops when the current state equals the target — so the pipeline can be safely re-run after any manual refinement.

3.2.6 Client-server communication with tRPC

The [tRPC](#) routers are defined under `libs/server/trpc/src/lib/routers/`, one router per domain (auth, pantry, onboarding, meal-plan, recipes, cooking-session and others). Each procedure is composed from one of three *builders*: `publicProcedure` (an alias for `procedure`, exposed anonymously, without authentication, defined in `libs/server/trpc/src/lib/trpc.ts`), `protectedProcedure` (requires a Firebase session verified in `createContext`) and `onboardedProcedure` (additionally requires `onboardingDone === true` on the user, otherwise it rejects the request with

PRECONDITION_FAILED), the last two declared in `libs/server/trpc/src/lib/middlewares/auth.ts` [27, 26]. The convention is a structural one: a grep for the builder used in a router immediately shows what surface the endpoint exposes, without having to inspect each locally composed *middleware* layer.

On the client side, the `tRPC` instance is created in `apps/web/src/trpc-client.ts`: the Firebase *ID token* is attached as the HTTP Authorization header through the `headers` option of the transport *link* (a reactive *signal* read on every request), and a dedicated `authAwareFetch` (i) proactively refreshes the token when it expires in less than five minutes, (ii) retries the request once with a new token on a 401 response, (iii) resolves relative URLs to absolute form in the `SSR` context (Node.js has no implicit base for `fetch('/api/...')`). In this configuration the `tRPC` procedures return RxJS Observables, and the `Angular` consumers choose between `from(...).pipe(...)` for a reactive stream and `await firstValueFrom(from(...))` for a single result [27].

3.2.7 Auxiliary h3 routes and the Redis infrastructure

Besides the `tRPC` handler (`apps/web/src/server/routes/api/trpc/[trpc].ts`), the application exposes two additional h3 routes: `/api/health` for the *healthcheck* and `/api/internal/bench-db` for the `CockroachDB` latency benchmark (used in the evaluation section). A single h3 *middleware* (`apps/web/src/server/middleware/security.ts`) covers two cross-cutting requirements: an explicit 404 for static files that are not found (`/assets/*`, `/fonts/*` and others) — to avoid the situation in which `SSR` would return 200 with HTML for a *bundle* replaced between deploys — and rejecting with 404 the usual automated scanning patterns (`/wp-admin`, `.env`, `.git`, `.php` extensions). Security headers are applied at two levels. At the network *edge*, through `vercel.json`, `HSTS` is applied with a one-year duration and `includeSubDomains`, which forces HTTPS access across the entire domain. The `CSP` policy is issued per request by a dedicated `Nitro` plugin (`apps/web/src/server/plugins/csp.ts`): for each HTML response, the plugin generates a unique *nonce*, attaches it to the application's inline scripts and includes it in the header, keeping `script-src` strict (the own origin plus the nonce) without allowing the execution of arbitrary inline code; the remaining sources stay limited to the own origin, with a *frame* exception for the Firebase authentication domain.

The Redis component lives in `apps/web/src/server/infra/` and is designed to tolerate unavailability. `getRedis()` is a singleton with a 60-second *circuit breaker*: after a failed connection, the following calls *fail fast* instead of recreating the client [21]. On the read path (`groceries.list`), a utility function silently catches Redis failures and returns the empty set, so that the page renders without the “bought” state; on the write path, the response includes a `persisted: false` field if Redis is unavailable, allowing the client to keep the *optimistic*

update and display a notification. Rate limiting (*rate-limiting*, `rateLimitFixedWindow`) uses an INCR + EXPIRE fixed window and is applied both to the *benchmark* route (10 requests per window) and to the [trRPC](#) handler: 10 requests for registration (`auth.register`) and 300 for the other procedures. In line with the unavailability-tolerance principle described above, rate limiting is *fail-open* — if Redis does not respond, the request is let through instead of being rejected, so that a failure of the *cache* layer does not block authentication or writes. Also to protect the free-tier Redis instance, a weekly scheduled task (*cron*) calls the `/api/cron/redis-keepalive` route, keeping it active to prevent the auto-suspension cycle.

3.2.8 Logging and observability

The [logging](#) layer is isolated in `libs/server/logger/`: a single [Pino](#) instance is created with an environment-dependent transport — `pino-pretty` with *colorize* and a short timestamp in development, raw JSON in production ([Vercel](#)) [17]. On the production path, the instance applies an explicit list of redacted fields (*redact list*: `password`, `*.token`, `authorization`) to prevent credentials from leaking into the logs.

Besides the global logging instance, `createLogger(namespace, bindings)` returns a child *logger* with a *namespace* label from a fixed set (`audit`, `http`, `trpc`, `prisma`) plus additional *bindings* — for example `createLogger('trpc', { router: 'pantry' })` produces `ns: 'trpc'` alongside `router: 'pantry'` on every emitted entry —, and `createAudit(bindings)` returns a logger specialized for audit events (with the `audit: true` flag applied automatically). Messages are always structured — `logger.info({ userId, action }, 'message')` — not string interpolations, so that the resulting logs can be filtered directly on the aggregation platform.

On top of the logging layer, error monitoring in production is provided by [Sentry](#), integrated symmetrically on the client and the server and inactive (*no-op*) until an ingestion key ([DSN](#)) is configured, so that development environments and deployments prior to the configuration remain untouched. On the client, a global `ErrorHandler` captures unhandled errors, and a `reportError helper` isolated in `libs/shared/observability/` is called at the error-handling points in the *stores* — the `catchError` blocks that consume the error to turn it into interface state —, making otherwise silent failures visible as well. On the server, the `wrap()` function in `libs/server/logger/` becomes the single capture point: the `error` and `fatal` methods forward the `err` field to `Sentry.captureException` before delegating to [Pino](#), so that every `log.error({ err })` call in the [trRPC](#) routers reports automatically, without instrumenting each error point. The `@sentry/node` client is initialized only once, at server startup, through a [Nitro plugin](#) that also registers a *hook* for unhandled errors, filtered to 5xx codes so as not to report 4xx client errors. The symmetry is deliberate: unhandled errors are caught by the global *handlers* on both ends, and handled ones by the two single capture points — `reportError`

on the client, `wrap()` on the server. Besides errors, the client also collects performance transactions through a *tracing* integration with a sampling rate of 10 %, enough to observe the pages' vital metrics without exceeding the free-tier quota. At build time, the `@sentry/vite-plugin` *plugin* uploads the *source maps* to [Sentry](#) and removes them from the delivered *bundle*: generated in *hidden* mode, without the `sourceMappingURL` comment, they are never served to the client, and each deployment is tagged with a *release* equal to the *Git commit hash*, so that a production error points exactly to the *deploy* that introduced it [23].

3.2.9 Execution flow of a request

The flow summarized in Section 3.1.2 unfolds in three stages.

Stage 1 (initial rendering). The HTTP request reaches the [Analog](#) server-side runtime (based on [Nitro](#)), and the [Angular](#) instance is executed on the server to produce the initial HTML [1, 29].

Stage 2 (querying the persistence layer). The [Angular](#) components trigger calls to the [tRPC](#) procedures of the server-side layer; each procedure validates its inputs with a [Zod](#) schema derived from the [Prisma](#) schema and uses `ctx.prisma` to query [CockroachDB](#) [27, 33, 5, 18]. The results propagate back through the [tRPC](#) layer to the components that take part in the initial rendering.

Stage 3 (hydration and subsequent interaction). The rendered HTML reaches the browser, and the [Angular](#) instance takes over through *hydration* — the DOM tree produced on the server is reused, and the events captured in the meantime are replayed by `withEventReplay()` [1]. From then on, subsequent interactions call the [tRPC](#) procedures directly from the browser, without a new *rendering* on the server [27]. Figure 4 shows this flow as a sequence diagram.

3.2.10 Testing and validation

The [Vitest](#) suite is orchestrated by `vitest.workspace.ts` at the root of the repository, which automatically collects all the `vite.config.*` and `vitest.config.*` files from the [Nx](#) projects [32, 31]. The specification files (*specs*, `*.spec.ts`) are co-located with the tested code, both in the *feature* libraries and in the server libraries (for example `pantry.router.spec.ts` next to `pantry.router.ts`). Granularity is the guiding principle: `nx affected -t test` runs only the specs touched by the current changes.

At the level of the server libraries, the [tRPC](#) procedures are tested in isolation, without starting an HTTP server and without a real database: each spec builds a fake context in which `ctx.prisma` is a test double with `vi.fn()` methods declared exactly for the operations touched by the procedure, and the procedure is invoked through `appRouter.createCaller(ctx)` [27, 32]. Since the [Prisma](#) client reaches the procedures through the context, not through a direct

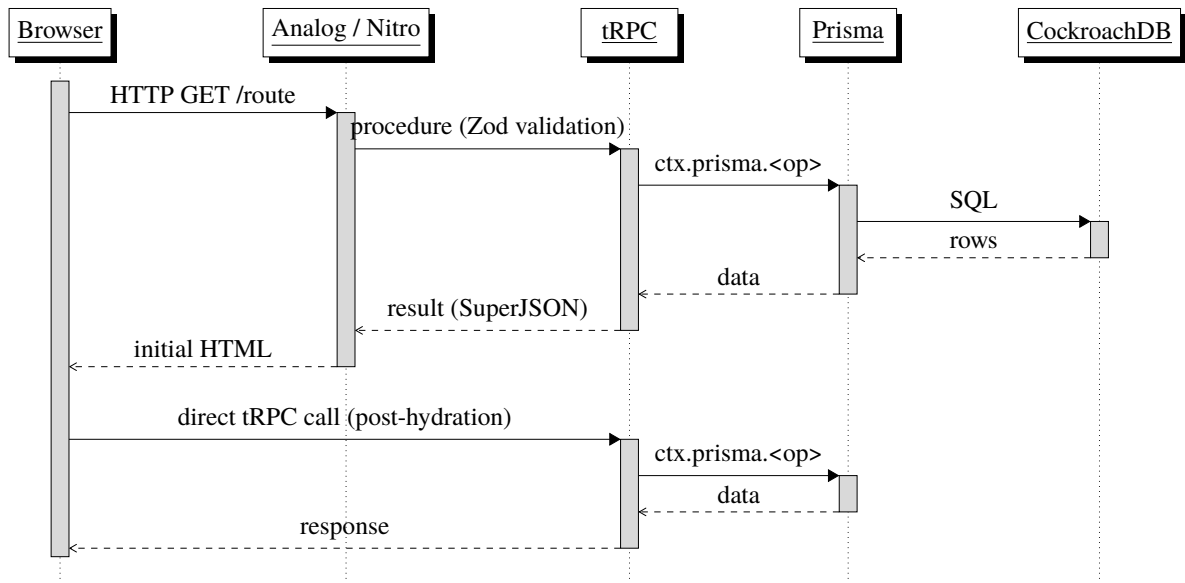


Figure 4: Sequence diagram of a request. **Initial rendering (SSR):** the HTTP request reaches the [Analog/Nitro](#) runtime, which queries the persistence layer through the [tRPC](#) (with [Zod](#) validation) → [Prisma](#) → [CockroachDB](#) layers and returns the initial HTML. **Subsequent interaction:** after hydration, calls go directly from the browser to [tRPC](#), without a new server-side rendering.

import, the context takes over the isolation role that a *repository* layer would have had: its absence (Section 3.2.2) does not sacrifice testability, it only moves the injection point into the context.

The [Playwright](#) suite lives at `apps/web-e2e/` and has its own `global-setup.ts` that warms up all the application’s routes before the first *worker* process starts — a measure needed to avoid the time constraints at the first start of [Vite](#) in the development environment [13]. The critical flows (authentication, registration, completing *onboarding*, adding to the pantry, viewing the planner) go through the interface in Chromium; protected routes use `waitUntil: 'commit'` to avoid the `ERR_ABORTED` error produced by [Angular](#) redirects during *hydration*.

The discipline most visible in practice applies to refactorings that change the dependencies injected into a shared component: the tests of the library in question keep passing (the local *stubs*, test stand-ins, are updated), but the specs of the dependent modules may fail with `NullInjectorError`. To detect these regressions, `nx affected -t test` is run, which follows the [Nx](#) dependency graph and runs everything that is affected [15].

3.2.11 Development and deployment

Environment configuration is centralized in the [Vercel](#) dashboard, separately for production, preview and development [30]. Locally, `vercel env pull` synchronizes the values of the current environment into a `.env.local` (excluded from version control through `.gitignore`), removing the need for any secret to be versioned in Git.

Promotion between environments follows the `dev` → `test` → `main` flow. The `dev` branch feeds the development *Preview* environment; the `test` branch triggers the [E2E](#) suite on the testing *Preview* environment before promotion to `main`, and the `main` branch feeds the production *deploy* in the `fra1` region (Frankfurt) [\[30\]](#).

The production build runs `nx build web` with `NODE_OPTIONS=-max-old-space-size=6144`. The default V8 heap limit is insufficient for bundling the entire [Angular](#) Material stack in the [Nitro](#) context: the phases that transform the abstract syntax tree (AST) and generate the *source maps* accumulate large volumes of temporary objects, and the build fails with JavaScript heap out of memory before completing. Raising the limit to 6 GB provides the necessary headroom for both phases. [Prisma Client](#) is regenerated in the automated *pipeline* — the `generate` target is a dependency of the `build` target —, so that the [TypeScript](#) client and the [Zod](#) schemas derived from `schema.prisma` are always in sync with the data model before the consumers are compiled [\[19\]](#). Since [Prisma 7](#) no longer ships a platform-dependent native engine (the client runs entirely in JavaScript through a *driver adapter*), regeneration concerns only the typed code, not binaries specific to the execution environment.

3.2.12 Architectural patterns discovered during implementation

Besides the structural principles established *a priori* (Section 2.2), implementing the [Kookio](#) application exposed three architectural patterns that were not predictable from the initial choices, but that consolidated into generalizable lessons for any [PAN](#) project. Unlike the intrinsic limitations of the stack (Section 4.2), the patterns described here are reusable *solutions*, codified in the project’s guidelines repository to prevent the same problem from recurring:

- **Authenticated SSR loaders require client-side hydration of authenticated data.** When first written, the `*.server.ts` loaders co-located with the dynamic routes called authenticated [tRPC](#) procedures directly through `appRouter.createCaller(ctx)`, assuming that the Firebase session from `createContext` is available both on the first [SSR](#) and on subsequent client-side navigation. The assumption was false: on client-side navigation, [Analog](#) executes the *loader* as an ordinary `fetch` that does not propagate the `Authorization` header with the Firebase *ID token*, and the authenticated procedures respond with 401. The correct pattern, codified in the repository: the *loader* loads only public data (*recipes.byId*, *about page content*), while authenticated data (the availability of ingredients in the pantry, the “saved” state of a recipe, the user’s preferences) is hydrated afterwards, from the browser, through the [tRPC](#) client with `authAwareFetch`. In terms of general applicability, the pattern comes down to the observation that [SSR](#) and *Bearer token* authentication do not coexist for free in any hybrid meta-framework — any reader evaluating [PAN](#) for their own project should audit this dimension before relying on [SSR loaders](#) for sensitive data.

- **Route names in file-based routers must be semantic, not numeric.** When building the page for the 500 error code, the natural choice was a `/500` route (parallel to the already working `/404`). At runtime, the route did not activate: the server responded with 200 and the HTML of the home page. The cause, uncovered through gradual investigation, was a quirk of the [Analog](#) file-based router: *all-numeric* segments are interpreted as invalid dynamic parameters, and the *matcher* silently falls through the list of routes without signaling a configuration error. The correct pattern, codified in the repository: *semantically* named routes (`/server-error`, `/not-found`, `/maintenance`) instead of numeric codes. In terms of general applicability, the pattern comes down to the observation that any file-based routing convention must separate *numeric codes* (HTTP protocol artifacts) from *route names* (semantic identifiers for the user) — even if, in the world of protocols, the two seem interchangeable.
- **A single defensible heuristic for estimating quantities, two consumers.** The offline data cleaning pipeline (the *seed* stage) and the runtime logic (pantry decrementing, shopping list derivation) initially consumed almost identical conversion heuristics, written separately — a manual `INGREDIENT_PIECE_GRAMS` table at *seed* time and their own *carve-outs* at runtime —, which forked every extension and, over time, produced behavioral divergences. Consolidating the two into a single source of truth (the pair of pure functions `toGrams/fromGrams` and the `IngredientCatalog.pieceGrams` column in the database schema) is detailed in Section 3.3.3. In terms of general applicability, the pattern comes down to a design rule: *one defensible heuristic, two or more consumers, never one heuristic per consumer*. A critique of the estimate (for example, “the density of water is a poor approximation for oils”) applies to all consumers at once; an improvement propagates the same way.

The three patterns share a common structure: each emerged from *implementation friction* (the [SSR loader](#) that responded with 401, the `/500` route that did not activate, two almost identical heuristics written separately), and the solution was not adding functionality, but *repositioning* the already existing piece at a level closer to the real invariant of the problem. From the standpoint of architectural auditing, this discovery pattern — *friction* → *lesson* → *codification* — has been observed in the software engineering literature as a characteristic of evolutionary systems, in which the correct invariant is learned from implementation experience, not imposed by the initial design.

3.3 Functional flows and product algorithms

The previous sections described how the architectural layer — [Analog](#), [tRPC](#), [Prisma](#) — materializes the [PAN](#) architecture at the code level. This section goes one level lower, to the *product*

algorithms that realize the application’s value proposition: progressive filtering of recipes based on the pantry inventory, transactional decrementing of quantities after cooking, and automatic derivation of the shopping list from the planned slots. The three flows converge into a single *estimation algorithm* that unifies the data cleaning pipeline (described in Section 3.2.4) with the production runtime logic. Progressive filtering of recipes based on the pantry inventory (Section 3.3.1) and automatic derivation of the shopping list (Section 3.3.4) operationalize two of the prevention strategies identified by Schanes et al. [22]: cooking with what is already stored at home and checking the inventory before shopping.

3.3.1 Progressive recipe filtering

On the `/recipes/match` route, the user sees the recipe catalog partitioned into three levels, labeled in the interface *Alright* (all non-negligible ingredients are present in the pantry), *Almost* (one to three non-negligible ingredients are missing) and *All* (all public recipes, unfiltered). The `trpc` procedure `recipes.match` receives a `level` field with exactly these three values (`'Alright'`, `'Almost'`, `'All'`), along with an optional cursor and a `limit` (default 20, maximum 50), and returns a page of recipes with the matching metadata the interface needs: the number of missing ingredients (`missingCount`), the names of the first three of them (`missingNames`) and the recipe’s total number of ingredients (`totalIngredients`). The page also carries a `nextCursor` for pagination and a `partialScan` flag used by the client to automatically continue a scan interrupted by the examination budget.

The algorithm is, in essence, a *set difference* between the recipe’s ingredients and the contents of the user’s pantry. For each recipe in the PUBLIC catalog, the procedure iterates through the list of `RecipeIngredients` and skips the ingredients with a negligible unit (`TO_TASTE`, `TO_SERVE`, `GARNISH`, `PINCH`) — salt, pepper, a parsley garnish and oil cannot block a recipe from the *Alright* level just because they are missing from the explicit pantry records. The remaining ingredients are looked up in a map built from `pantryItem.findMany({ where: { userId } })`, compared at the level of *existence* (the presence of the ingredient in the pantry, regardless of quantity). In the version delivered with this thesis, the quantity comparison is not performed; a rare false positive is preferred over unit logic that would duplicate the pipeline described in Section 3.3.3.

The decision to keep the matching logic simple — *presence*, not *sufficient quantity* — was taken deliberately: the cost of a false positive is low (a recipe appears as cookable although the user has an insufficient quantity of an ingredient), whereas the quantitative variant would duplicate the unit conversion logic of the grams pipeline (Section 3.3.3) at the matching boundary, with no proportional gain in perceived accuracy. For future development, the quantitative variant is built directly on top of that pipeline — ingredients are normalized to grams before comparison, and the

Alright level requires the pantry to cover the recipe’s consumption in grams for all non-negligible ingredients.

3.3.2 The cooking session and transactional decrementing

On the `/cook/:recipeId` route, the user goes through the recipe’s steps and, at the end, a dialog offers a list of pre-filled quantities (proportional to the selected number of servings) for confirmation. On submission, the client calls `cookingSession.complete`, which decrements the pantry in a single [Prisma](#) transaction (`$transaction`). For each confirmed (`ingredientCatalogId`, `consumedQuantity`), the procedure looks up the corresponding row by the compound key (`userId`, `ingredientCatalogId`) (the `@@unique` constraint declared in [Section 3.2.4](#)), subtracts the consumed quantity from the existing quantity and — this is where the estimation algorithm described in the next section comes in — deletes the row if the remaining quantity drops below a minimum threshold (0.5 g, the noise threshold of a pinch of salt). The entire operation runs atomically: either all the decrements go through and `cookingSession.completedAt` is updated, or nothing is applied.

Two implementation details are worth mentioning. First, the decrementing is done internally in grams (not in the display unit of the pantry row), to allow consumption in units different from the storage unit — for example, a recipe calls for 200 g of flour, but the user entered the flour as 0.5 kg. The conversion back to the display unit happens at the end, before `prisma.update`. Second, if the user indicated a consumption greater than the quantity available in the pantry (an input mistake, or a recipe with a larger quantity entered later), the quantity is clamped to zero and the row is deleted, instead of receiving a negative value. These robustness behaviors at the edge of the data flow were validated through eighteen unit specification cases for `cookingSession.complete` at the time of the grams pipeline refactoring (28 May 2026).

3.3.3 The grams pipeline: one heuristic, two consumers

The two previous flows — progressive filtering ([Section 3.3.1](#)) and the cooking session ([Section 3.3.2](#)) — depend on the same practical question: *how do we convert quantities between heterogeneous units?* The recipe calls for 1 cup of sugar and the pantry holds 500 g; the recipe calls for 2 lemons, but the pantry has only 1 piece. A classic approach would separate each case (*carve-out*): a converter for weights, a converter for volumes, a special rule for pieces. Over time, such layering produces branches that grow with every unit added.

The solution adopted in [Kookio](#) is a pair of pure functions — `toGrams` and `fromGrams` — that normalize all quantities to grams before any operation and convert back to the display unit only at the end. The pipeline handles four classes of units — weight, volume, pieces and taste/garnish —, each with an explicit conversion rule; the per-class rules, the `allowDefault`

option for pieces without a known weight and the implementation of the `toGrams` function are detailed in Section [A.2](#).

A single case is worth keeping here, since it is a design decision, not a mere conversion detail: the taste and garnish units (`TO_TASTE`, `TO_SERVE`, `GARNISH`) are treated as 0 g and excluded from the shopping list. The 0 g value is a consciously chosen modeling convention, not an unknown: it honors the intention of the recipe’s author (the quantity is deliberately left “to taste”), avoids downstream noise (staple seasonings decremented on every cooking session or listed for shopping are not useful information in an application focused on the waste of bulk food) and keeps a neutral position with respect to dietary intake, which the application is not in a position to prescribe.

The 0 g convention does, however, have a known limitation that follows directly from it: staple ingredients seasoned “to taste” (salt, oil, pepper) are not tracked at the quantity level, so their gradual depletion — through small, repeated uses — is not signaled to the user. The right solution to this gap is not assigning an arbitrary non-zero value (which would reintroduce the noise described above and would produce, in the absence of a warning infrastructure, data that nothing consumes), but a dedicated mechanism for tracking consumables, based on a state (*in stock / running low / out of stock*) marked by the user, not on estimated mass. Such a module — foreshadowed by the manual “running low” marker already present in the pantry interface — is described as a direction for future development in Section [4.3](#).

The structural decision this pipeline represents goes beyond the implementation detail: it is a *unification* between two processes that previously used similar but separate heuristics. When generating the initial data sets (*seeds*), an offline cleaning pipeline imported the *TheMealDB* catalog and estimated the missing weights using a manual `INGREDIENT_PIECE_GRAMS` table built for this stage. At runtime, pantry decrementing (Section [3.3.2](#)) and shopping list derivation (Section [3.3.4](#)) required the same conversions, but through separately written code. By extracting the `toGrams/fromGrams` functions into a common library (`libs/shared/utility`) and promoting the manual table to the `IngredientCatalog.pieceGrams` column, both processes now consume the same source of truth. From an auditing standpoint, a critique of the estimate (for example, “the density of water is a poor approximation for oils”) applies to both consumers at once; from an evolution standpoint, an improvement (a layer of per-category ingredient densities) propagates the same way. This design lesson is formulated as a generalizable architectural pattern in Section [3.2.12](#).

The `toGrams/fromGrams` pair is a textbook application of information hiding [\[16\]](#): the changeable design decision — the concrete way of converting between heterogeneous units (weight factors, approximating volumes through the density of water, the `pieceGrams` values for piece-type units) — is isolated behind a stable interface, and the two consumers (the offline *seed* pipeline and the runtime logic) depend exclusively on this interface, not on the implementation. The rule formulated above — one defensible heuristic, two or more consumers — restates the

criterion according to which a module’s interface should reveal as little as possible about its internal mechanism. The benefit is the one described by Parnas: a critique of the estimate (the density of water as a poor approximation for oils) or an improvement (a layer of per-category densities) applies to all consumers at once, because there is a single place where the decision is materialized.

3.3.4 Deriving the shopping list

On the `/groceries` route, the shopping list appears as a *set difference on ingredient identity* between the ingredients required by the planned slots and the contents of the pantry. The `groceries.list` procedure expands each slot with a recipe in the user’s plan (all slots with a non-null `recipeId`) into the ingredients of the corresponding recipe, scales the quantities by the ratio `slot servings / recipe servings`, aggregates by `ingredientCatalogId` and completely removes any ingredient for which a row already exists in the pantry — the removal comparison is made on *presence*, not on quantity. Ingredients with a *flavour-only* unit (`TO_TASTE`, `TO_SERVE`, `GARNISH`) are excluded from the list (salt, pepper and garnishes do not appear on the shopping list; `PINCH` is kept, having a non-zero mass (0.5 g)).

Reconciliation in grams does not happen when filtering the list, but when merging with the pantry: at the end, the `groceries.addBoughtToPantry` mutation transfers the items checked as “bought” directly into the pantry, merging them with any existing rows for the same ingredient. This merge uses the grams pipeline from Section 3.3.3 — it normalizes the quantities to grams, sums them and converts back to the storage unit of the existing row (or of the new item, if the row is missing). Piece-type items without a populated `pieceGrams` consume `DEFAULT_PIECE_GRAMS = 100 g` through the `allowDefault` option, a *graceful degradation* solution preferable to failing the operation because of a single incomplete ingredient.

The “bought” state of each item is persisted in [Redis](#) (Section 3.2.7), allowing the user to continue the list on different *devices* within the key’s expiration window.

3.4 Methodology and evaluation

3.4.1 Experimental setup

All measurements were taken in the [Vercel Preview](#) environment (the `fra1` region, Frankfurt), with [CockroachDB](#) Serverless and [Redis Cloud](#) in the same region; the stack runs [Analog](#) on top of [Nitro](#), with [SSR](#) on the first request and [hydration](#) on the client. The data set comes from deterministic test data sets (`fixtures`, `libs/shared/mock/`) and from the initialization scripts (`seed`, `libs/prisma-client/seed-data/catalog.json`, `recipes.json`).

All the measurement procedures are versioned in the project’s repository: the benchmark route is defined in `apps/web/src/server/routes/api/internal/bench-db.get.ts`, the [CI](#) flows are in `.github/workflows/`, and the configuration of the [E2E](#) suite is in `apps/web-e2e/playwright.config.ts`. Each dimension is reproduced through a dedicated script in `scripts/`: persistence latency through `npm run bench:preview`, test coverage through `npm run bench:coverage`, and front-end performance — the [Lighthouse](#) scores and the image payload — through `npm run bench:lighthouse` and `npm run bench:images`, respectively; the scripts that query the [Vercel](#) environment require the corresponding access credentials.

3.4.2 Indicators and procedures

Latency at the persistence layer. The internal `/api/internal/bench-db` route executes, on each call, $N = 100$ successive read queries against [CockroachDB](#) and [Redis](#), respectively, reporting the arithmetic mean, the 50th percentile ([p50](#)) and the 95th percentile ([p95](#)). The cold/warm distinction operates at the level of the *serverless* function invocation: the first call pays for the cold start of the instance and of the *connection pool* to the database, while subsequent calls hit the already warmed-up instance. The cold call is reported separately from the median of the warm calls.

Reliability of the end-to-end ([E2E](#)) tests. The pass rate of the [Playwright](#) suite on Chromium, measured over the last ten runs on the `test` branch. WebKit also passes completely, and Firefox runs most of the tests, except a few authentication tests (marked `@no-firefox`), excluded because of a synchronization race at the level of the *fetch* requests in the local development environment. In [CI](#) only Chromium runs systematically; Firefox and WebKit are covered in manual runs before promotions to `main`.

Front-end performance. [Lighthouse](#) is run on the home page of the application published at `kookio.app`, in the desktop and mobile profiles (the latter with network and CPU *throttling*), reporting the median of several runs for the performance score and for the [LCP](#), [CLS](#) and [TBT](#) metrics. Separately, the payload reduction of the image optimization is quantified: for each of the 594 images in the catalog, the original JPEG file is compared with the [AVIF](#) variant delivered by the [Vercel](#) optimizer at the maximum configured width ($w = 960$) — a conservative bound, since smaller responsive widths save even more.

Unit test coverage. Line coverage is reported over the entire source code of the libraries (*all-source*: files not covered by any test are counted as 0%, not omitted), aggregated from the per-project [Vitest](#) reports. The web application is excluded from this figure, because its test

environment times out under coverage instrumentation; its user flows are validated by the [E2E](#) suite.

3.4.3 Results

Table 2: Summary of the results across the three measured dimensions.

Indicator	Value
<i>Latency at the persistence layer ($N = 100$ probes/call, median over warm calls)</i>	
CockroachDB (warm) — mean	1.79 ms
CockroachDB (warm) — p50	1.71 ms
CockroachDB (warm) — p95	2.07 ms
CockroachDB (cold start) — p50 / p95	2.6 / 6.4 ms
Redis (warm) — mean	0.59 ms
Redis (warm) — p50	0.56 ms
Redis (warm) — p95	0.77 ms
<i>Front-end performance (Lighthouse, median)</i>	
Desktop score	93
Mobile score (with <i>throttling</i>)	58
LCP desktop / mobile	1.5 / 8.3 s
CLS (desktop / mobile)	0.055 / 0
Image payload reduction (594 images, $w = 960$)	−60.5 %
<i>Coverage and testing</i>	
Line coverage (library source code)	82.2 %
Unit tests (Vitest)	873 / 96 files
E2E tests (Playwright)	14 / 7 flows
E2E pass rate on Chromium	100 %

The results summarized in Table 2 confirm, within the limits explained in Section 3.4.4, the viability of the [PAN](#) architecture along the three evaluation axes: millisecond latencies at the persistence layer, solid front-end performance on desktop (score 93, zero [CLS](#)) with an acknowledged gap in the throttled mobile profile, a reduction of approximately 60 % in the image payload across the entire catalog, a coverage of 82.2 % of the library code and complete reliability of the critical end-to-end flows on the reference browser.

3.4.4 Discussion and limitations

All measurements come from the *Preview* environment, in the fra1 region, without concurrent traffic — so they do not capture the stack’s behavior under load or in a multi-region setup.

The [E2E](#) suite runs only on Chromium in [CI](#), and the Firefox and WebKit test matrices are covered manually; automated promotion of the two would increase confidence in *cross-browser* compatibility. In addition, both [CockroachDB](#) and [Redis](#) run on the free tiers (with usage quotas and an *auto-suspension* cycle for [Redis](#)) and are measured without concurrent traffic, which makes the reported latencies representative of a single-user regime, not of production at scale. Generalizing these results requires dedicated load tests and repeated measurements in a stable production environment. Moreover, these latencies are dominated by the free-tier managed providers — the network time to the `fra1` region and the free-quota limitations —, so they characterize the deployment infrastructure, not the [PAN](#) stack’s own *overhead*. Isolating this overhead would require a local, containerized benchmark (for example with Docker), without the network latency of the cloud providers — a measurement direction left for future work.

Three limitations specific to the new measurements are worth pointing out. First, the persistence latencies vary noticeably between measurement sets — the `p50` median for [CockroachDB](#) oscillated between approximately 1.4 and 2.6 ms —, a consequence of the free tiers, the single region and the variable load; the value reported in [Table 2](#) is the median of a set of warm calls, not a constant. Second, the [Lighthouse](#) score on the mobile profile reflects the aggressive *throttling* applied by the tool (slow network and a four-times slower CPU), not the performance perceived on a real device — the desktop profile remains in the good range. Finally, the 82.2 % coverage concerns exclusively the library code, the web application being excluded for the reasons explained in [Section 3.4.2](#).

4 Conclusions

I proposed the [PAN](#) architecture and validated it by implementing [Kookio](#), a full-stack [web](#) application built entirely within the [TypeScript](#) ecosystem on this model. Starting from the preference for [TypeScript](#) motivated in [Chapter 2](#), I translated this preference into a concrete stack: a single [Analog](#) application under `apps/web`, four groups of [Nx](#) libraries (`feature`, `prisma`, `server`, `shared`) with an acyclic topology enforced by `enforce-module-boundaries` in [CI](#), and an end-to-end [Analog](#) → *[tRPC](#)* → [Prisma](#) flow described in [Section 3.1.2](#).

Adopting the [PAN](#) architecture made it possible to eliminate the classic adaptation layers between frontend and backend. The *[tRPC](#)* procedures call `ctx.prisma` directly, without a *repository* layer ([Section 3.2.2](#)), and the [Zod](#) schemas consumed by [Angular](#) are derived from `schema.prisma` through `prisma-zod-generator` — renaming a field propagates at compile time to all consumers. The public `@kookio/prisma-client` entry point is safe to run in the browser (it exports only types and [Zod](#) schemas), while the `PrismaClient` instance is isolated under the `@kookio/prisma-client/server` sub-path, blocked by a dedicated ESLint rule in frontend code.

The combination of **Angular** and **Analog** delivered **SSR** for all the application's routes, configured in a single place (`apps/web/vite.config.ts`). The application-level **SSR** configuration is minimal — `provideFileRouter()`, `provideHttpClient(...)` and `provideClientHydration(withEventReplay())` —, and the discipline identified in practice (Section 3.2.3) is that every import added to `app.config.ts` carries an amplified cost on the critical path of every route, which is why the heavy Material components remain imported from the level of the page that uses them.

At the persistence layer, **Prisma** on top of **CockroachDB** provided benefits directly observable at the code level. Automatically generating the **TypeScript** client and the **Zod** schemas from `schema.prisma` reduces the data contract to a single source of truth; the compound constraints declared as `@@unique (MealSlot on userId-date-slotType, PantryItem on userId-ingredientCatalogId)` are enforced at the database level, and the corresponding lookup queries use the automatically generated compound-key form. To tolerate the retries that **CockroachDB** may require under the default *serializable* isolation, the **Prisma** instance is extended with an automatic *retry-on-conflict* for the codes P2034, P2036 and 40001 (plus the *restart transaction* message); all write operations (`create`, `update`, `upsert`, `delete` and the `*Many` variants) go through this *wrapper*, declared only once.

The testing process confirmed the value of combining two distinct levels. The **Vitest** suite (orchestrated by `vitest.workspace.ts` at the root of the repository) covers isolated logic through specification files co-located with the tested code, and `nx affected -t test` runs only the specs touched by the current change. The **Playwright** suite at `apps/web-e2e/` goes through the critical flows (`authentication`, *onboarding*, `pantry`, `planner`) in Chromium and achieved a pass rate of 100 % over the ten reference runs (Table 2). Two practices consolidated from the concrete **CI** cycles: warming up the routes in `global-setup.ts` before the first *worker* process starts, and `waitUntil: 'commit'` for protected routes, in response to the `ERR_ABORTED` error produced by **Angular** redirects during hydration.

With respect to the objectives stated in Chapter 1, I delivered a **minimum viable product (MVP)** that covers the application's core flows: `pantry` management, daily meal planning, shopping list derivation and user *onboarding*. The **monorepo** structure based on vertical slices (`libs/feature/auth`, `onboarding`, `pantry`, `plan`, `recipes...`), in which `libs/feature/X` cannot import from `libs/feature/Y` — a rule checked in **CI** by `enforce-module-boundaries` —, allows extension with new flows as additional libraries, without structural changes to the existing application.

Overall, **Kookio** shows that the **PAN** architecture can work coherently for a real full-stack **web** application: the latencies measured on the `/api/internal/bench-db` route (median over warm calls of 1.71 ms for **CockroachDB** and 0.56 ms for **Redis** — Table 2), the front-end performance (a **Lighthouse** score of 93 on desktop and a reduction of about 60 % in the image payload across the entire catalog), the 73.2 % coverage of the library code and the 100 % pass

rate of the [E2E](#) suite on Chromium support this model within the limits explained in Section 3.4.4 — the *Preview* environment, the single `fra1` region, free tiers for [CockroachDB](#) and [Redis](#), the *cross-browser* test matrix covered manually and the absence of concurrent traffic, with the reported latencies reflecting a sequential regime, not performance at scale.

The main contribution of this thesis is the proposal and empirical validation of the [PAN](#) architecture as a coherent model for full-stack [web](#) applications in the [Angular/TypeScript](#) ecosystem, demonstrated through a working case study in the domain of reducing household food waste.

4.1 Architectural decisions

During the implementation, several initial decisions were refined, eliminated or reconfigured in response to implementation friction. The nine architectural decisions documented in the project’s repository fall into three distinct categories, summarized in Table 3: four *simplifications* (eliminating systems that were built but had no real consumer), three *unifications* (patterns discovered from friction and codified as rules, presented in detail in Section 3.2.12) and two *design decisions* (interface choices revised after implementation).

In the context of this bachelor’s thesis, each decision represents an *adjustment of the invariant* that the [PAN](#) architecture exposed when meeting a concrete use case. The simplifications (categories 1–4) show that a type-coherent stack makes it possible to quickly identify dead code by searching for *consumer types*, not just import paths: if a [tRPC](#) procedure, a *guard* or a [Prisma](#) model is not referenced anywhere in the frontend, removing it is safe at compile time. The unifications (categories 5–7) illustrate that the [PAN](#) architecture exposes friction that is not visible in the theoretical presentation of the stack (Section 2.3), but that arises from the interaction between components at runtime. The design decisions (categories 8–9) are independent of [PAN](#) as such, but show that the structural discipline of the stack does not prevent revising interface proposals — the removed components were organized in their own libraries and could be taken out without touching the rest of the application.

The reflective tone of these decisions — refined or removed when meeting a concrete case — has a direct precedent in the founding article on information hiding: in the section “*Improvement in Circular Shift Module*”, Parnas analyzes his own module and finds that its interface revealed more information than necessary (it imposed an ordering of the shifts), needlessly restricting the class of possible implementations — an error he explicitly classifies as a design error [16]. Documenting one’s own revised decisions is therefore not a sign of project immaturity, but a practice present in the very text that introduced the principle.

Table 3: The nine documented architectural decisions, organized by category.

Category	Decision	Generalizable lesson
<i>Simplifications — system built, zero real consumers, removal</i>		
1	Removing biometric authentication (<i>WebAuthn</i>)	The system was built in response to an anticipated, not an observed, requirement; no interface actually consumed it.
2	Removing the role infrastructure (role-based access control)	The only consumer was a stub page for the administrator; the removal dropped the <code>Role</code> enum, the <code>User.role</code> column and the <code>roleProcedure builder</code> .
3	Flattening the <code>MealPlan</code> aggregate	The intermediate aggregate had no real consumer; the <code>UI</code> worked directly with <code>MealSlot</code> on (<code>userId</code> , <code>date</code> , <code>slotType</code>). The removal reduced the number of models in the schema by one.
4	Removing the <code>core/</code> library group	The <i>feature flags</i> subsystem and the configuration/security concerns were fully wired, but had no consumer (the router was not mounted, the <i>guard</i> protected no route); the removal deleted the <code>core/config</code> and <code>core/security</code> libraries plus the <code>APP_TRPC</code> token, and <code>core/logging</code> was absorbed into <code>server/logger</code> — the taxonomy went from five to four groups.
<i>Unifications — implementation friction, codified lesson (Section 3.2.12)</i>		
5	<code>SSR</code> loaders require client-side hydration of authenticated data	The Firebase <i>Bearer token</i> is not propagated through <code>fetch</code> on client-side navigation; loaders call only public procedures.
6	Routes named semantically, not through numeric codes	The <code>Analog</code> file-based router silently falls through on <i>all-numeric</i> segments; the solution: <code>/server-error</code> instead of <code>/500</code> .
7	A single heuristic for estimating quantities, two consumers	The offline cleaning pipeline and the runtime logic now consume the same <code>toGrams/fromGrams</code> pair (Section 3.3.3).
<i>Design decisions — initial proposal, validated through later use</i>		
8	<i>Cook now</i> button moved from the <i>FAB UI</i> component to the topbar	The <i>FAB (floating action button)</i> component took up space without offering a visibility advantage over a button in the topbar.
9	<i>Quick actions</i> widget removed from the home page	The proposed quick actions were redundant with the bottom navigation bar plus the <i>Cook now</i> button; the removal reduced the visual noise of the home page.

4.2 Limitations and trade-offs of the architecture

Adopting [PAN](#) as an architecture is not without trade-offs. This subsection lists them explicitly, so that a reader evaluating [PAN](#) for their own project can make an informed choice:

- **The adoption curve.** [PAN](#) requires solid knowledge of [Angular](#) and of advanced [TypeScript](#) (*generics*, *conditional types*, *mapped types*). For a team without [Angular](#) experience, learning [PAN](#) costs significantly more than adopting a conventional [Next.js](#).
- **Analog maturity.** At the time of writing, [Analog](#) is a project with a community of a few thousand users and no major adoption in the corporate environment. The risk that certain production problems have no documented answer is real. The mitigation used in this thesis was selecting components that are mature individually ([Angular](#), [Nitro](#)) and explicitly accepting the risk for the [Analog](#) orchestration layer.
- **Coupling to TypeScript.** [tRPC](#) assumes that both ends of the communication are [TypeScript](#). [PAN](#) is not a viable option for applications that must expose a public API consumed by clients in other languages (Java, Go, Python). In such cases, GraphQL or OpenAPI remain more suitable choices.
- **Moderate vendor dependency.** Although [PAN](#) can be deployed on any Node.js-compatible infrastructure ([Nitro](#) supports [Vercel](#), Netlify, AWS, Cloudflare, Bun, Deno), the integration with [Vercel](#) is the most well-trodden. Migrating to another provider is possible, but not effortless.
- **CockroachDB at a small scale.** For applications under 100,000 requests per day, the operational complexity of [CockroachDB](#) (the Raft consensus algorithm, the default *serializable* isolation that requires retry logic) is disproportionate compared to standard [PostgreSQL](#). [PAN](#) with classic [PostgreSQL](#) would be a simpler variant for applications that do not need real horizontal scaling.
- **Limited learning resources.** Community content (Stack Overflow, blog posts, courses) for [PAN](#) as such does not exist — it exists only for the individual components. A developer facing a problem specific to the [Analog](#) + [tRPC](#) + [Prisma](#) + [Nx](#) integration has to build their own solution from first principles.
- **Gains given up by forgoing distribution.** The same review [24] identifies as major operational gains of microservices independent horizontal scalability (gain #1, 31 studies), independent *deployment* of components and the freedom to choose a distinct technology *stack* for each service. [PAN](#) explicitly gives up the first two: the application is released as a single unit, and real scaling under concurrent traffic has not been characterized

(Section 3.4.4). The third is the very negative of one of its founding principles — end-to-end [TypeScript](#) uniformity (Section 2.2) is chosen precisely at the expense of per-component technological freedom. For a project that requires independent scaling of subsystems with divergent load profiles, or that must integrate services written in different languages, these trade-offs would tip the balance back towards a distributed architecture.

[PAN](#) is, therefore, not a universally applicable architecture. It is an architecture for small teams with solid [Angular](#) knowledge building modern, monolithic (in the sense of single-unit release) and data-centric [TypeScript](#) applications. In this solution space, [PAN](#) offers a technological coherence and a structural discipline that React-based alternatives reach only through *ad hoc* choices and conventions.

4.3 Future directions

The natural directions of evolution start from the limitations identified in Section 3.4.4 and from the architectural trade-offs discussed in Section 4.2. At the infrastructure level, promoting [CockroachDB](#) and [Redis](#) to paid tiers, together with enabling a second region beyond `fra1` ([CockroachDB](#) natively supports cross-zone replication, which can be enabled without changes to the application code), would make it possible to characterize the stack under concurrent traffic. In the area of testing, systematically including the Firefox and WebKit test matrices in [CI](#) would strengthen confidence in *cross-browser* compatibility beyond Chromium. In terms of functionality, the existing [Redis](#) infrastructure (used for now to mark bought items on the shopping list and to rate-limit the benchmark route — Section 3.2.7) can be extended with an application-level *caching* layer for public pages. Recommending recipes based on the pantry inventory is already covered by progressive filtering (Section 3.3.1), and leveraging the preferences from *onboarding* is partially implemented — optional filtering by the preferred categories and culinary areas. Dietary restrictions are already collected and persisted in the *onboarding* flow (a dedicated step writes them into `UserDietaryRestriction`), but they do not yet take part in the recommendation: what remains to be added is labeling the recipes with the restrictions they satisfy — a `Recipe–DietaryRestriction` relationship currently absent from the schema, derivable from each recipe’s ingredients — after which the filter would plug into the same optional mechanism used for categories and areas. Normalizing the ingredient catalog also contributes to matching accuracy: matching between recipe and pantry is done on the identity of the catalog ingredient, but some of the ingredients in the imported recipes use generic names (for example “flour” or “potatoes”) that have no direct counterpart in the catalog — which contains only qualified variants, such as *All purpose flour* or *Plain Flour* — so the recipes that call for them cannot reach the *Alright* level starting from the pantry’s contents. Extending the table of equivalences (*aliases*) applied when importing the data (Section 3.2.5) would reduce these mismatches and increase the proportion of fully covered recipes. Along the same axis, a module

for tracking consumables would address the limitation identified in Section 3.3.3: the staple ingredients seasoned “to taste” (salt, oil, pepper), which are currently not quantified, would be tracked at the level of a state (*in stock / running low / out of stock*) marked by the user, and the items flagged as out of stock would automatically feed the shopping list, without resorting to mass estimates for quantities intentionally left to the cook’s discretion.

Beyond the directions specific to the *Kookio* product, the PAN architecture itself remains extensible along several axes not explored in the case study:

- **Real-time communication.** *tRPC* supports *subscriptions* over WebSocket or Server-Sent Events; a natural line of evolution is integration with a message broker (*Redis* Pub/Sub, *NATS*) for notifications to authenticated clients.
- **Functions at the network edge (*edge functions*).** *Nitro* allows partial deployment at the *edge* (*Vercel* Edge Functions, *Cloudflare* Workers). *tRPC* procedures that do not require *Prisma* can be moved to the *edge* to reduce geographical latency.
- **Mobile applications.** The *Angular* stack supports native mobile applications through *Capacitor* or *NativeScript*; the shared and *prisma* libraries of a PAN project are directly reusable between a *web* application and a mobile one.
- **Internationalization.** *Angular* i18n combined with multilingual *Analog* routing (routes parameterized by a language prefix) covers the case of multilingual applications without architectural changes.

The acyclic topology of the *monorepo* allows these extensions to be made incrementally: a new library in `libs/feature/` with its own *tRPC* router is added without touching the rest of the application.

Appendix A Demonstration of the *Kookio* application

This appendix documents the main functional flow of the *Kookio* application, as evidence that the *minimum viable product* (MVP) described in Section 3 is operational from end to end. The screenshots capture a real user’s journey — from the pantry inventory, through meal planning and the shopping list, to the cooking session that decrements the stock. The final section relocates here the implementation detail of the unit conversion pipeline, kept in the body of the thesis only at the level of the design decision (Section 3.3.3).

A.1 The main flow: from setup to cooking

The demonstrated journey covers the application’s value chain, from the initial account setup to the automatic stock update after cooking: the user sets their preferences, declares what they have in the pantry, sees what they can cook, plans their meals, derives the shopping list for what is missing and, after cooking, the pantry is decremented.

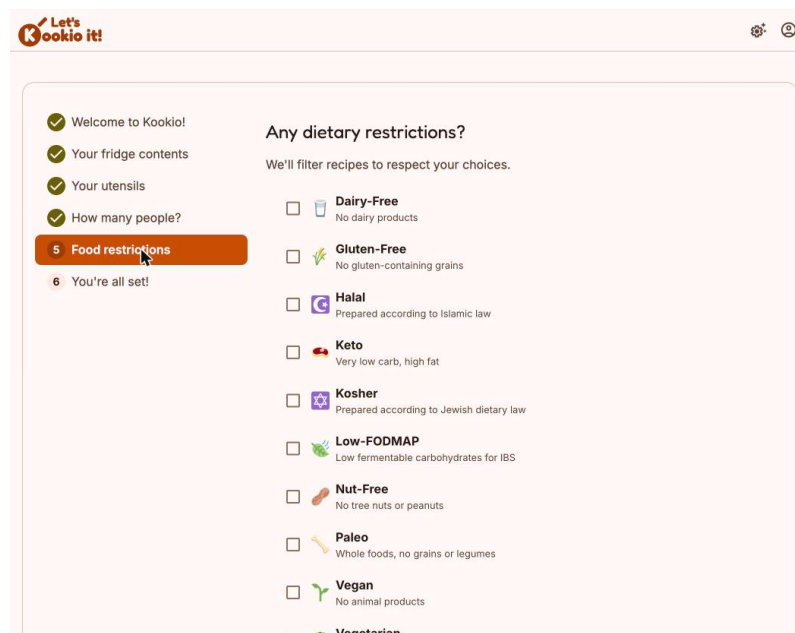


Figure 5: Onboarding (/onboarding): the initial setup steps — culinary preferences (categories and areas), dietary restrictions and available utensils — persisted on completion and used later for filtering recipes.

A.2 Details of the unit conversion pipeline

The body of the thesis (Section 3.3.3) keeps the unit conversion pipeline at the level of the design decision — the pair of pure functions `toGrams/fromGrams` as an application of information hiding. The concrete detail of the conversion rules is relocated here, being an implementation

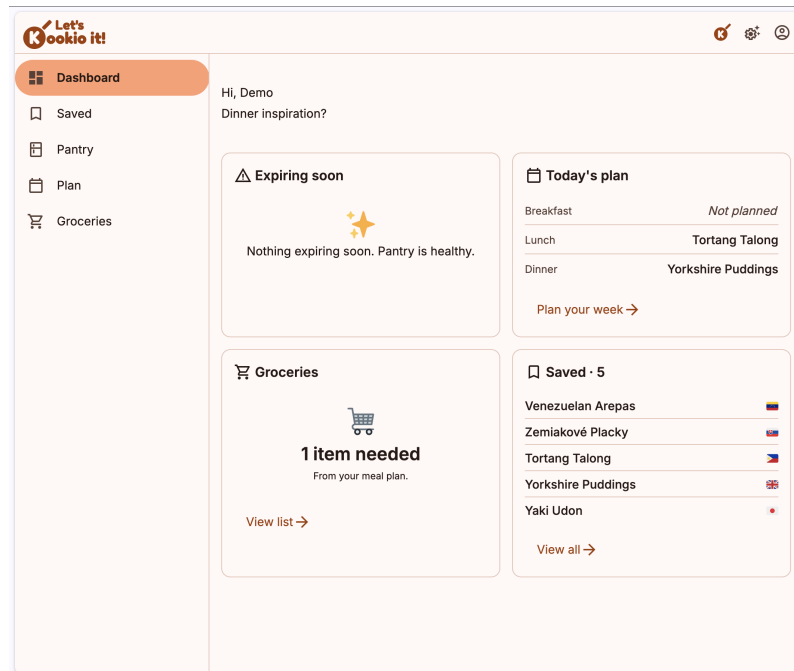


Figure 6: The dashboard (/dashboard): the entry point after authentication, with access to the application's flows and a summary of the current state.

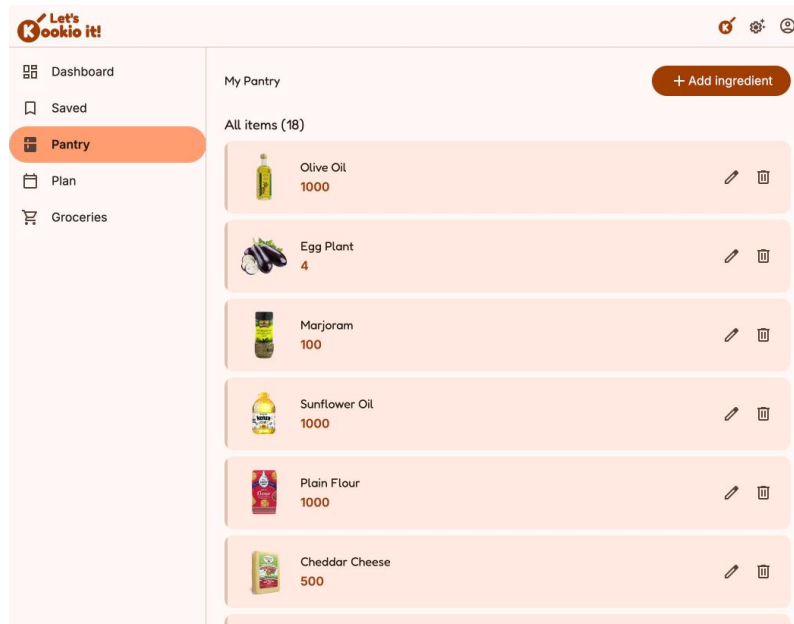


Figure 7: The pantry (/pantry): the user's inventory, with each ingredient stored in its display unit. This is the starting point of the flow — the source of truth for what is available at home.

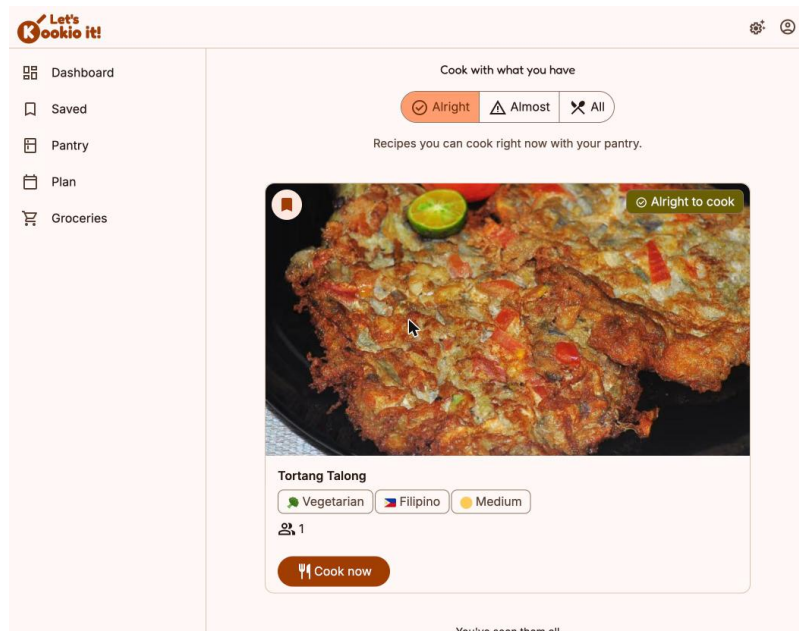


Figure 8: Progressive filtering (/recipes/match): the catalog partitioned by the degree of coverage from the pantry (Section 3.3.1). The *Alright* level lists the fully covered recipes, *Almost* those missing one to three ingredients.

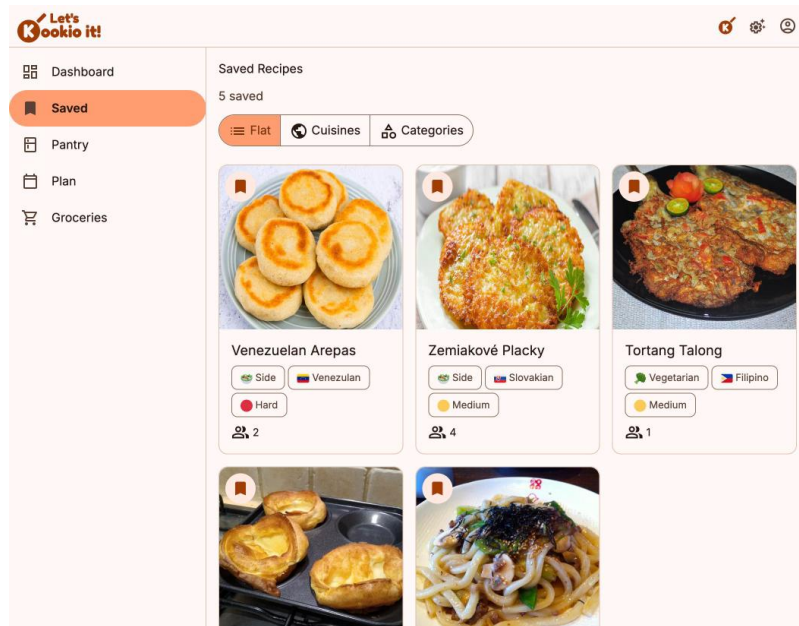


Figure 9: Saved recipes (/recipes/saved): the collection of recipes marked by the user for later access, independent of the degree of coverage from the pantry.

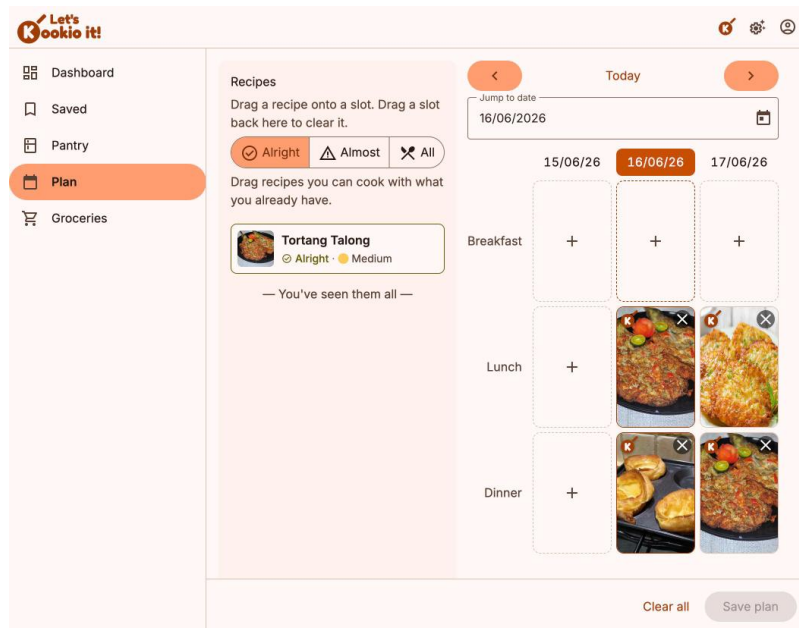


Figure 10: The planner (/plan): recipes are assigned to (date, meal type) slots, modeled as MealSlot with the @@unique constraint described in Section 3.2.4.

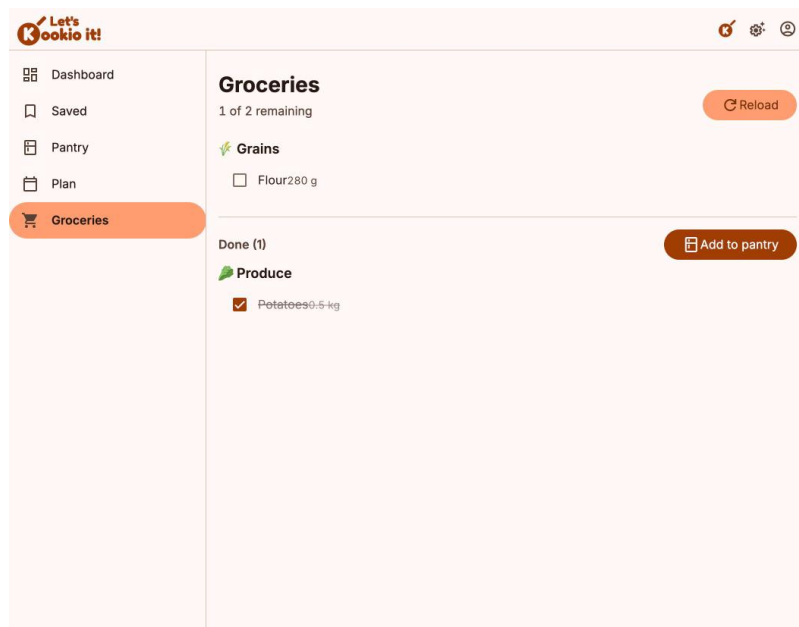


Figure 11: The shopping list (/groceries): the set difference between the ingredients of the planned slots and the contents of the pantry (Section 3.3.4). Items checked as “bought” can be merged back into the pantry.

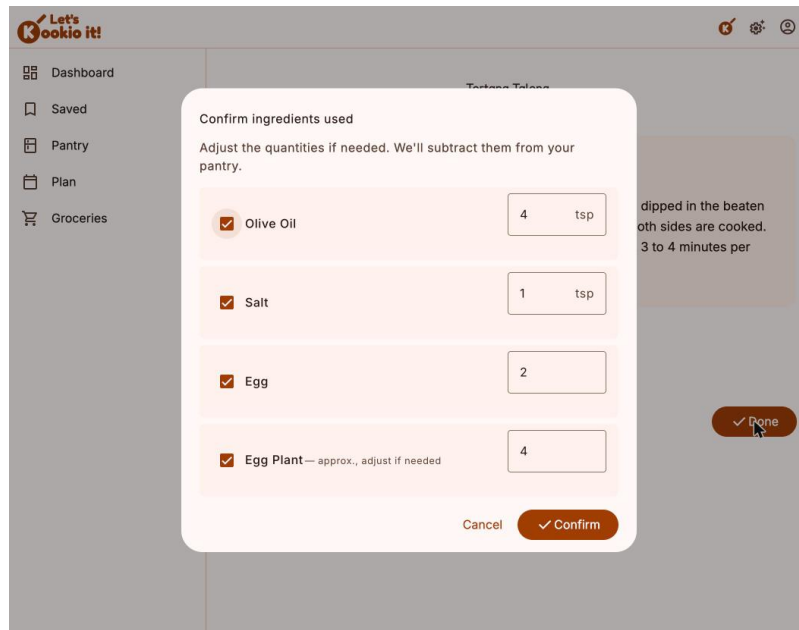


Figure 12: The cooking session (`/cook/:recipeId`): at the end, the dialog offers the pre-filled consumed quantities, and the confirmation triggers the transactional decrementing of the pantry (Section 3.3.2).

detail specific to the application, useful as a reference, but not necessary for the architectural argument.

The pipeline handles four classes of units, each with an explicit rule:

- **Weight** (G, KG, OZ, LB): direct conversion through constant factors.
- **Volume** (ML, L, TSP, TBSP, CUP): conversion through the density of water ($1 \text{ ml} \approx 1 \text{ g}$), an acceptable approximation for the ingredients commonly used in home cooking.
- **Pieces** (PIECE, SLICE, CLOVE, BUNCH, OTHER): conversion through the optional `IngredientCatalog.pieceGrams` column (`Float?`), populated with 32 values (egg: 60, garlic clove: 5, lemon: 100 etc.).
- **Taste and garnish** (TO_TASTE, TO_SERVE, GARNISH): treated as 0 g and excluded from the shopping list. The distinction from PINCH is not one of size, but of nature: PINCH is a quantity specified by the recipe’s author (a pinch), whereas the taste and garnish units express an instruction — “to taste”, “to serve” — whose quantity is deliberately left to the discretion of whoever is cooking. The 0 g value is therefore a consciously chosen modeling convention, not an unknown: it honors the author’s intention (assigning a gram-equivalent would override a decision explicitly left open), avoids downstream noise (half a gram of salt decremented on every cooking session, or staple seasonings listed for shopping, are not useful information in an application focused on the waste of bulk food) and keeps a neutral position with respect to dietary intake, which the application is not in a position to

prescribe. For PINCH, on the other hand, a convention sets 0.5 g per unit — a small but real mass, enough to allow sub-pinch rows to be deleted from the pantry.

For piece-type ingredients without a populated `pieceGrams` value, the function returns `null`, signaling to the caller that it cannot produce an estimate. An optional `allowDefault` parameter changes this behavior: when explicitly enabled, the function returns `DEFAULT_PIECE_GRAMS = 100 g` per piece. The 100 g value is a rough approximation, chosen as a plausible order of magnitude for an arbitrary piece, not an empirically measured average; its role is that of a safety net, not of an exact estimate — which is why it is not applied by default, but only at the caller's explicit request. This option protects the callers that prefer an explicit error (the cooking session, where an unfounded consumption would corrupt the inventory) from silently accepting unreliable estimates, while allowing the consumers that prefer graceful degradation (the shopping list, where an approximate estimate is preferable to a total failure) to continue processing.

Listing 1 presents the `toGrams` function. The complete implementation, together with the inverse `fromGrams`, is located in `libs/shared/utility/src/lib/labeling/measure-unit-labels.ts`.

Listing 1: The `toGrams` function — normalizing quantities to grams.

```
export const toGrams = (
  amount: number,
  unit: MeasureUnit,
  pieceGrams: number | null = null,
  opts: GramsConversionOptions = {},
): number | null => {
  if (FLAVOUR_ONLY_UNITS.has(unit)) return 0;

  const fixed = UNIT_TO_GRAMS_FIXED[unit];
  if (fixed !== undefined) return amount * fixed;

  // Piece-class: PIECE, SLICE, CLOVE, BUNCH, OTHER.
  const piece =
    pieceGrams !== null && pieceGrams > 0
      ? pieceGrams
      : opts.allowDefault
        ? DEFAULT_PIECE_GRAMS
        : null;
  if (piece === null) return null;
  return amount * piece;
};
```

Appendix B The data model

The complete database schema — the 20 [Prisma](#) models — is shown in Figure 13 as an entity-relationship diagram. The diagram is generated automatically from `schema.prisma` (with *prisma-dbml-generator*) and rendered with *dbdocs.io*, so it stays in sync with the actual data model. Primary keys are marked with the key symbol, foreign keys with the link symbol, and the connectors between tables show the relationships between entities; the junction tables (for example `UserDietaryRestriction`, `RecipeCollectionEntry`) are placed next to the entities they link.

Glossary of Terms

Analog full-stack meta-framework for [Angular](#), with [SSR](#) support and file-based routing; used for [web](#) applications

Angular platform and framework for [web](#) applications (SPA/[SSR](#))

Lighthouse Google tool for automated performance auditing of [web](#) pages, which reports *Core Web Vitals* metrics ([LCP](#), [CLS](#), [FCP](#), [TBT](#))

Nitro universal runtime (UnJS) for server routes and server-side rendering ([SSR](#))

Nx build and orchestration system for JavaScript/TypeScript monorepos

Prisma [ORM](#) library for TypeScript/Node.js

Zod schema-based validation library for TypeScript/JavaScript

CockroachDB distributed (NewSQL) database management system, compatible at the protocol level with [PostgreSQL](#), with automatic replication and partitioning for resilience and horizontal scaling

composite action a type of reusable action in GitHub Actions that groups several steps (run/uses) into a single action; it is distributed as a directory with an `action.yml` file, accepts *inputs*, *outputs* and *secrets* and can be called from a [workflow](#) through `uses:.` Useful for eliminating duplication (DRY) in [CI/CD](#) pipelines; common limitations: they do not run in their own containers and cannot define *services*.

hydration the process by which client-side code attaches interactive behavior to the *markup* generated through [SSR](#); it allows the transition from the initial server-side rendering to [CSR](#) interactions without reloading the page

Kookio full-stack [web](#) application used as a case study in this thesis for pantry management, daily meal planning and automatic derivation of the shopping list, built on the [PAN](#) architecture

logging collecting and recording application events (structured messages by level, correlation through identifiers, export to aggregation systems) for debugging and observability

meta-framework a framework that extends a base framework with full-stack capabilities (e.g. file-based routing, [SSR/SSG](#), integration with [tRPC](#)). In this thesis, [Analog](#) extends [Angular](#) on top of the [Nitro](#) runtime.

monorepo a code organization strategy in which several applications and libraries coexist in a single repository, with unified orchestration of builds, tests and [CI/CD](#); in the context of this thesis it is managed with Nx

Next.js React framework oriented towards hybrid rendering ([SSR/SSG](#)), often associated with JAMstack

Pino performance-oriented logging library for Node.js (JSON logs, low-overhead output)

Playwright library for browser automation and [E2E](#) testing, with support for Chromium, Firefox and WebKit, headless/GUI execution and tools such as the trace viewer

PostgreSQL open-source relational database management system, with extensive support for the SQL standard, ACID transactions and extensibility through user-defined types and functions

Prisma Client TypeScript database client, generated automatically by [Prisma](#) from the `schema.prisma` file; it provides a typed API for SQL operations (CRUD, filters, relations, transactions) on the defined models. Starting with [Prisma 7](#), the client runs entirely in JavaScript (without a native Rust engine): the connection goes through a *driver adapter* (`@prisma/adapter-pg`), and regenerating with `npx prisma generate` produces only typed code, independent of the execution platform.

Redis an *in-memory* key–value data store, with advanced structures (lists, sets, hashes, sorted sets, *streams*) and very low latency; frequently used for caching, session storage, queues/messaging (Pub/Sub, Streams), rate-limiting and *leaderboards*. It supports optional persistence (RDB/AOF), replication and *cluster* mode for partitioning and, as a rule, complements a primary SQL database in microservice and *serverless* architectures.

rendering the process by which an application generates HTML (and interface updates) from code and data. On the modern web it can take place on the server ([SSR](#)), at build time through [SSG](#)/prerendering, or in the browser ([CSR](#)); as a rule, it is followed by [hydration](#) for interactivity. In [Analog](#), the rendering strategy can be chosen per route.

Sentry production error monitoring platform, which aggregates, groups and alerts on the exceptions captured from the client and the server

TypeScript a typed superset of the JavaScript language that introduces static types and is transpiled to JavaScript

USES a relation between modules defined by Parnas (1972): a module A is in the *USES* relation with module B if the correct functioning of A depends on the presence of a correct version of B. Parnas writes it in capital letters as a formal term and requires this relation to be a loop-free (acyclic) partial ordering, a property that allows functional subsets of the system to be built by “cutting off” the upper levels

Vercel cloud platform for building, previewing and deploying [web](#) applications, with [CI/CD](#) integration, per-environment variables and support for [SSR](#), *serverless* functions and *edge functions*

Vite build tool and development server for modern [web](#) applications, based on ECMAScript modules, with very fast startup and *hot module replacement* (HMR); it underpins [Vitest](#) and is used by [Analog](#) for [SSR](#) and builds

Vitest fast testing framework for the Vite ecosystem, aimed at unit and integration tests, with an API largely compatible with Jest and native TypeScript support

web the ecosystem of *hypertext* resources accessible over the Internet (World Wide Web); in this thesis, “web application” denotes an application accessed from the browser, with [SSR/SSG/CSR](#) rendering

workflow a declarative configuration describing a sequence of automated steps (build, testing, lint, deploy) triggered by events (e.g. push, pull request, schedule). In ecosystems such as GitHub Actions and GitLab [CI](#), a workflow is made up of jobs and steps, and can use matrices, caches, artifacts, secrets/variables and concurrency rules. In this project, the workflows orchestrate [CI/CD](#) and align execution with the [Nx](#) targets.

List of Acronyms

AVIF image format based on the AV1 codec

CI continuous integration

CI/CD continuous integration and continuous delivery/deployment

CLS Cumulative Layout Shift

CSP Content-Security-Policy

CSR client-side rendering

DSN Data Source Name

DTO data transfer object

E2E end-to-end testing

FCP First Contentful Paint

HSTS HTTP Strict-Transport-Security

LCP Largest Contentful Paint

MEAN MongoDB–Express–Angular–Node.js

MVP minimum viable product

ORM object–relational mapping

p50 50th percentile (median)

p95 95th percentile

PAN Prisma–Analog–Nx

PR pull request

SEO search engine optimization

SPA single-page application

SSG static site generation

SSR server-side rendering

TBT Total Blocking Time

tRPC TypeScript remote procedure call

UI user interface

References

- [1] AnalogJS Contributors. *Analog – The Full-Stack Meta-Framework for Angular*. Analog Documentation. Official Analog documentation; cited for SSR, file-based routing and the Nitro/tRPC integration in Angular applications. 2026. URL: <https://analogjs.org/docs/features/server-side-rendering> (visited on 05/18/2026).
- [2] AnalogJS Contributors. *Routing — File-Based Routing*. Analog Documentation. Official Analog documentation; cited for *filesystem-based* routing (`*.page.ts`, dynamic routes with parameters, co-locating server logic in `*.server.ts`). 2026. URL: <https://analogjs.org/docs/features/routing/overview> (visited on 05/18/2026).
- [3] Cockroach Labs. *CockroachDB Documentation*. CockroachDB Documentation. Official CockroachDB documentation; cited in the development plan for the analysis of horizontal scalability. 2026. URL: <https://www.cockroachlabs.com/docs/stable/postgresql-compatibility> (visited on 05/18/2026).
- [4] Devographics. *State of JavaScript 2024*. Annual Survey of the JavaScript Ecosystem. Annual survey of the JavaScript community (over 22,000 respondents); cited for the usage patterns of SSR (59%) and SSG (46%), TypeScript adoption and the complexity of frontend frameworks. 2024. URL: <https://2024.stateofjs.com/en-US/libraries/front-end-frameworks/> (visited on 05/18/2026).
- [5] Omar Dulaimi and contributors. *Prisma Zod Generator*. GitHub Repository. Cited for the automatic generation of Zod validation schemas from the Prisma models. 2026. URL: <https://github.com/omar-dulaimi/prisma-zod-generator> (visited on 05/18/2026).
- [6] European Parliament. *Food waste in Europe: facts, EU policies and 2030 targets*. European Parliament Topics. Official European Parliament overview; cited for the behavioral causes of food waste at consumer level (impulse buying, insufficient meal planning, poor inventory management) and for the 2030 targets. 2025. URL: <https://www.europarl.europa.eu/topics/en/article/20240318ST019401/food-waste-in-europe-facts-eu-policies-and-2030-targets> (visited on 05/17/2026).
- [7] Eurostat. *Food waste: 132 kg per inhabitant in the EU in 2022*. Eurostat News Release. Official Eurostat news release; cited for data on food waste per inhabitant in the European Union (132 kg/inhabitant in 2022, 59.2 million tonnes in total) and for the share of households (54%) in total waste. 2024. URL: <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20240927-2> (visited on 05/17/2026).
- [8] Zheng Gao, Christian Bird, and Earl T. Barr. “To Type or Not to Type: Quantifying Detectable Bugs in JavaScript.” In: *Proceedings of the 39th International Conference on Software Engineering (ICSE)*. An empirical study that quantifies, on a sample of 400 public bugs from GitHub projects, the proportion detectable through static typing (Flow and TypeScript); cited for the empirical grounding of the advantages of typing and for the taxonomy of undetectable bugs. IEEE, 2017, pp. 758–769. doi: [10.1109/ICSE.2017.75](https://doi.org/10.1109/ICSE.2017.75).
- [9] Google Angular Team. *Angular Material — Components with Built-in Accessibility*. Angular Material/CDK Documentation. Official Angular Material documentation; cited for the UI component set and for the Angular CDK, consistent with respect to accessibility (*ally*). 2026. URL: <https://material.angular.dev/> (visited on 05/18/2026).

- [10] Google Angular Team. *Signals Overview*. Angular Documentation. Official Angular documentation; cited for the reactivity model (Signals) used in standalone components and in the context of server-side rendering (SSR). 2026. URL: <https://angular.dev/guide/signals> (visited on 05/18/2026).
- [11] JetBrains. *JavaScript and TypeScript Trends 2024*. Insights From the Developer Ecosystem Survey. Annual JetBrains survey; cited for the growth of TypeScript adoption (12% in 2017, 35% in 2024) and the share of predominantly TypeScript developers (67%). 2024. URL: <https://blog.jetbrains.com/webstorm/2024/02/js-and-ts-trends-2024/> (visited on 05/17/2026).
- [12] Valeri Karpov. *The MEAN Stack: MongoDB, ExpressJS, AngularJS and Node.js*. MongoDB Blog. The original article in which Valeri Karpov introduced the term “MEAN stack”; cited as a historical reference for the traditional architecture of the JavaScript ecosystem and for the conceptual parallel with the PAN architecture proposed in this thesis. Apr. 2013. URL: <http://blog.mongodb.org/post/49262866911/the-mean-stack-mongodb-expressjs-angularjs-and> (visited on 05/17/2026).
- [13] Microsoft Playwright Team. *Testing with Playwright — Getting Started*. Playwright Documentation. Official Playwright documentation; cited for the end-to-end testing adopted in June–July 2025. 2026. URL: <https://playwright.dev/docs/test-global-setup-teardown> (visited on 05/18/2026).
- [14] Nrwl Team. *Enforce Module Boundaries — ESLint Rule for Architectural Constraints*. Nx Documentation. Official Nx documentation; cited for the @nx/enforce-module-boundaries rule, which rejects in CI the imports that violate the topology between library groups. 2026. URL: <https://nx.dev/docs/features/enforce-module-boundaries> (visited on 05/18/2026).
- [15] Nrwl Team. *What is Nx?* Nx Documentation. Official Nx documentation; cited for monorepo orchestration, affected runs, caching and the project graph. 2026. URL: <https://nx.dev/docs/features/ci-features/affected> (visited on 05/18/2026).
- [16] David L. Parnas. “On the Criteria To Be Used in Decomposing Systems into Modules.” In: *Communications of the ACM* 15.12 (1972). The founding article of the information hiding principle and of modular decomposition; cited for the acyclic “uses” relation, for the critique of flowchart-based decomposition and for hiding changeable design decisions., pp. 1053–1058. DOI: [10.1145/361598.361623](https://doi.org/10.1145/361598.361623).
- [17] Pino Authors. *Pino — Logger for Node.js*. Pino Documentation. Official Pino documentation; cited for the centralized logging used in both the backend and the frontend. 2026. URL: <https://getpino.io/#/docs/transporters> (visited on 05/18/2026).
- [18] Prisma Labs. *Connecting Prisma to CockroachDB*. Prisma ORM Documentation. Official Prisma documentation; cited for the integration with CockroachDB through the PostgreSQL wire protocol and for the particularities of distributed support. 2026. URL: <https://www.prisma.io/docs/orm/overview/databases/cockroachdb> (visited on 05/18/2026).
- [19] Prisma Labs. *Prisma Client — Generating the Typed Client and Queries*. Prisma ORM Documentation. Official Prisma documentation; cited for generating the TypeScript client from schema.prisma, managing versioned migrations and type inference for each model. 2026. URL: <https://www.prisma.io/docs/orm/prisma-client> (visited on 05/18/2026).

- [20] Prisma Labs. *Transactions and Batch Queries*. Prisma Client Documentation. Official Prisma documentation; cited for `prisma.$transaction`, handling *serializable* conflicts (code P2034) and automatic retry strategies in distributed databases. 2026. URL: <https://www.prisma.io/docs/orm/prisma-client/queries/transactions> (visited on 05/18/2026).
- [21] Redis. *Redis Documentation*. Redis Documentation. Official Redis documentation; cited in the development plan for the caching and query-session management strategies. 2026. URL: <https://redis.io/docs/latest/> (visited on 05/18/2026).
- [22] Karin Schanes, Karin Dobernig, and Burcu Gözet. “Food waste matters — A systematic review of household food waste practices and their policy implications.” In: *Journal of Cleaner Production* 182 (2018). A systematic review (60 empirical studies) of household food waste practices; cited for the behavioral causes at consumer level and for the role of planning, inventory checks and digital applications in prevention., pp. 978–991. DOI: [10.1016/j.jclepro.2018.02.030](https://doi.org/10.1016/j.jclepro.2018.02.030).
- [23] Sentry Authors. *Sentry — Application Error Monitoring*. Sentry Documentation. Official Sentry documentation; cited for production error monitoring, integrated symmetrically on the client and the server. 2026. URL: <https://docs.sentry.io/> (visited on 06/13/2026).
- [24] Jacopo Soldani, Damian Andrew Tamburri, and Willem-Jan Van Den Heuvel. “The Pains and Gains of Microservices: A Systematic Grey Literature Review.” In: *Journal of Systems and Software* 146 (2018). A systematic review of the industrial literature (51 studies) on the pains and gains of the microservices architecture; cited for positioning the PAN architecture as a modular monolith in the space of trade-offs., pp. 215–232. DOI: [10.1016/j.jss.2018.09.082](https://doi.org/10.1016/j.jss.2018.09.082).
- [25] The PostgreSQL Global Development Group. *Official PostgreSQL Documentation*. PostgreSQL Documentation. Official PostgreSQL documentation; cited for the *wire protocol* that *CockroachDB* emulates for compatibility with the SQL tooling ecosystem. 2026. URL: <https://www.postgresql.org/docs/current/protocol.html> (visited on 05/18/2026).
- [26] The tRPC Project. *Middleware — Procedure Builders and Composing Authorization*. tRPC Documentation. Official tRPC documentation; cited for the compositional procedure builders (`publicProcedure`, `protectedProcedure`) and for identity verification in the context. 2026. URL: <https://trpc.io/docs/server/middlewares> (visited on 05/18/2026).
- [27] The tRPC Project. *tRPC Documentation — End-to-End Type-Safe APIs*. tRPC Documentation. Official tRPC documentation; cited for the type contracts between client and server and for the router/context structure. 2026. URL: <https://trpc.io/docs/server/procedures> (visited on 05/18/2026).
- [28] United Nations Environment Programme. *Food Waste Index Report 2024*. Official UNEP Report. UNEP report, co-authored by WRAP; cited for global data on household food waste (60% of total waste, 631 million tonnes in 2022) and for the SDG 12.3 target of halving food waste by 2030. 2024. URL: <https://www.unep.org/resources/publication/food-waste-index-report-2024> (visited on 05/17/2026).

- [29] UnJS Team. *Nitro: The Web Server Engine*. Nitro Documentation. Official Nitro documentation; cited for server-side execution, API routes and the deployment environments compatible with Analog SSR. 2026. URL: <https://nitro.build/guide> (visited on 05/18/2026).
- [30] Vercel. *Vercel CLI — Command Line Interface*. Vercel Documentation. Official documentation for managing projects and environment variables in Vercel. 2026. URL: <https://vercel.com/docs/cli/env> (visited on 05/18/2026).
- [31] Vite Team. *Vite Guide*. Vite Documentation. Official Vite documentation; cited for the build configuration shared by applications and libraries and for the integration with the Nx targets. 2026. URL: <https://vitejs.dev/guide/why.html> (visited on 05/18/2026).
- [32] Vitest Team. *Vitest Guide*. Vitest Documentation. Official Vitest documentation; cited for the configuration and use of unit tests, used in the project since August 2025. 2026. URL: <https://vitest.dev/guide/workspace.html> (visited on 05/18/2026).
- [33] Zod Authors. *Zod Documentation*. Zod Documentation. Official Zod documentation; cited for the validation of the schemas used together with the types generated by Prisma. 2026. URL: <https://zod.dev/?id=objects> (visited on 05/18/2026).